MIM UW · 2025/2026

Reinforcement Learning

Comprehensive Exam Answers

Generated June 16, 2026

Contents

Click any entry to jump to it.

★ Reinforcement Learning — Exam Answers

This document gives a thorough, self-contained answer to each exam question. Display formulas are typeset with LaTeX; in running text we use light symbols ($\mathbb{E}$ = expectation, π = policy, γ = discount, V = state-value, Q = action-value).

● Reinforcement-learning basics

1. What is RL formalism (MDP), policy, value function, rewards? Give examples.

The setting. Reinforcement Learning (RL) studies an *agent* that interacts with an *environment* over discrete time steps. At each step the agent observes a state, chooses an action, receives a scalar reward, and transitions to a new state. The goal is to act so as to maximize the **expected cumulative reward**.

Markov Decision Process (MDP). The standard formalism is a tuple (S, A, P, R, γ):

- **S** — set of states.
- **A** — set of actions.
- **P(s′ | s, a)** — transition dynamics: probability of landing in s′ after taking a in s. The *Markov property* holds: the next state depends only on the current state and action, not on the full history.
- **R(s, a)** (or R(s,a,s′)) — reward function, the immediate scalar feedback.
- **γ ∈ [0, 1]** — discount factor that trades off immediate vs. future reward.

The agent's objective is to maximize the **return**, the discounted sum of rewards from time t:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k \geq 0} \gamma^k R_{t+k+1}$$

Reading it term by term.

- G_t — the **return** from time t: the quantity we ultimately want to maximize, looking only *forward* from t.
- R_{t+1} — the reward that arrives one step *after* t (the first consequence of the action taken at t); R_{t+2} the next, and so on.
- $\gamma \in [0,1]$ — the **discount factor**. The weight γ^k shrinks a reward that arrives k steps later, so near rewards matter more than distant ones.
- The compact sum $\Sigma_{k \geq 0}\ \gamma^k R_{t+k+1}$ just collects every future reward, each scaled by how far away it is.

Intuition: γ near 0 makes the agent **myopic** (it cares only about the next reward); γ near 1 makes it **far-sighted**. γ < 1 also guarantees the infinite sum stays finite.

Policy (π). A policy maps states to actions. It can be *deterministic* a = π(s), or *stochastic* π(a | s) = probability of choosing a in s. The policy is what the agent actually controls and learns.

Value functions quantify "how good" a state or state-action pair is under a policy, by measuring expected return:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[\, G_t \mid S_t = s \,] \qquad Q^{\pi}(s,a) = \mathbb{E}_{\pi}[\, G_t \mid S_t = s,\ A_t = a \,]$$

Reading it term by term.

- $\mathbb{E}_{\pi}[\,\cdot\,]$ — an **average** taken assuming actions are drawn from policy π and states evolve under the environment's dynamics. Because the future is random, value is an *expected* (mean) return, not a single number.
- G_t — the return defined just above.
- "$\mid S_t = s$" — read "**given** we are in state s at time t".
- $V^{\pi}(s)$ averages the return over all the ways an episode can unfold *starting from s and acting with π*.
- $Q^{\pi}(s,a)$ is the same average, but the **first action is pinned to a** ("$\mid S_t=s, A_t=a$"); only *after* that first step do we follow π.

Intuition: V scores a **state**; Q scores a **state plus a committed first move**. Their sole difference is whether that first action is fixed in advance.

Reward is the immediate signal; **value** is the long-term expected return. A key principle (the *reward hypothesis*) is that any goal can be encoded as the maximization of expected cumulative reward.

Examples.

- **Grid world / maze**: states = cells, actions = {up, down, left, right}, reward = −1 per step and +10 at the goal; the agent learns the shortest path.
- **Chess / Go**: states = board positions, actions = legal moves, reward = +1 for a win, −1 for a loss, 0 otherwise.
- **Robot locomotion**: states = joint angles/velocities, actions = motor torques, reward = forward velocity − energy cost.
- **Recommender system**: state = user context, action = item to show, reward = click / watch-time.

2. What are the basic components of a reinforcement learning algorithm?

An RL agent is usually built from up to three components; different algorithm families emphasize different ones.

1. **Policy** — the agent's behavior function (the mapping from state to action). This is the core output of learning. May be explicit (e.g. policy-gradient methods store π directly) or implicit (e.g. greedy w.r.t. a value function).
2. **Value function** — a prediction of expected future reward, used to evaluate states/actions and to guide improvement of the policy.
3. **Model** — the agent's learned representation of the environment dynamics, $\hat{P}(s' \mid s,a)$ and $\hat{R}(s,a)$, used to *plan* (simulate outcomes). Agents with a model are *model-based*; those without are *model-free*.

Cross-cutting ingredients present in essentially every RL algorithm:

- **Return / objective** — what we maximize (discounted cumulative reward).
- **Exploration vs. exploitation** — a mechanism (ε-greedy, entropy bonus, optimism, Thompson sampling) to try new actions while exploiting known good ones.
- **Update rule** — how experience changes the policy/value/model (e.g. Bellman backups, gradient ascent on expected return).
- **Experience source** — interaction data: online streaming, batches, or a replay buffer.

A useful taxonomy: **value-based** (learn V/Q, derive π), **policy-based** (learn π directly), **actor-critic** (learn both), and **model-based** (learn dynamics and plan).

◆ Value-based methods

1. What is the relation of value function and a policy?

They are two sides of the same coin and are linked in **both** directions.

- **Policy → value (evaluation/prediction).** Given a policy π, its value function V^{π} (or Q^{π}) is *defined* as the expected return obtained by following π. Computing it is called *policy evaluation*.
- **Value → policy (improvement/control).** Given a value function we can derive a better policy by acting **greedily**: in each state pick the action with the highest value,

$$\pi'(s) = \arg\max_a Q^{\pi}(s, a)$$

Reading it term by term.

- π'(s) — the action the **improved** policy takes in state s.
- $\arg\max_a$ — "the action a that makes the following quantity the largest" (returns the *action*, not the value).
- $Q^{\pi}(s,a)$ — value of taking a in s and then following the old π.

Intuition: in every state, switch to whichever action the current value function rates highest. The policy-improvement theorem guarantees this can only make the policy better (or leave it unchanged).

The *policy improvement theorem* guarantees $V^{\pi'} \geq V^{\pi}$. Alternating evaluation and improvement (**generalized policy iteration**) converges to the **optimal** policy π* and value V*. At the optimum the policy is greedy with respect to its own value function — they are mutually consistent.

2. What is the difference between value function and q-function?

- **State-value $V^{\pi}(s)$** = expected return starting from state s and then following π. It answers "how good is this state?"
- **Action-value (q-function) $Q^{\pi}(s, a)$** = expected return starting from s, taking an arbitrary action a *now* (not necessarily the one π would pick), then following π. It answers "how good is this action in this state?"

They are related by:

$$V^{\pi}(s) = \sum_{a} \pi(a \mid s)\, Q^{\pi}(s,a) \qquad Q^{\pi}(s,a) = \sum_{s'} P(s' \mid s,a)\big[\, R + \gamma V^{\pi}(s') \,\big]$$

Reading it term by term.

Left (V from Q): - Σ_a π(a|s) Q^{π}(s,a) — weight each action's value by how often π chooses it, then sum. Averaging the action out of Q turns it back into the state value V.

Right (Q from V): - Σ_s, P(s'|s,a) — average over the next states the dynamics can produce after taking a in s. - R + γV^{π}(s') — the immediate reward plus the discounted value of wherever we land.

Intuition: V and Q are two views of the same thing. **Average Q over actions → V**; **push V one step forward through the model → Q**.

Why Q matters: to act greedily from V you need the model (argmax$_a$ Σ P(s′|s,a)[R+γV(s′)]), but from Q you can act greedily *model-free*: π(s) = argmax$_a$ Q(s,a). That is why control algorithms (SARSA, Q-learning, DQN) learn Q rather than V.

3. Formulate Bellman equation. How many are there?

Bellman equations express value recursively: the value of a state equals the immediate reward plus the discounted value of the successor.

Bellman *expectation* equations (for a fixed policy π):

$$V^{\pi}(s) = \sum_{a} \pi(a \mid s) \sum_{s'} P(s' \mid s,a)\big[\, R(s,a,s') + \gamma V^{\pi}(s') \,\big]$$

Reading it term by term.

- Outer Σ_a π(a|s) — average over the actions π might take in s.
- Inner Σ_s, P(s'|s,a) — average over the next states that action can lead to.
- R(s,a,s') + γV^{π}(s') — one step's reward, plus the discounted value of the successor state.

Intuition: "value now = average over (my action, the environment's response) of (reward now + discounted value next)." It is just the return definition written **recursively** — one step made explicit, all later steps folded into V^{π}(s'). Because there is no max, this equation is **linear** in the unknown values and can be solved directly.

$$Q^{\pi}(s,a) = \sum_{s'} P(s' \mid s,a)\Big[\, R + \gamma \sum_{a'} \pi(a' \mid s')\, Q^{\pi}(s',a') \,\Big]$$

Reading it term by term.

- Action a is already fixed, so we only average over next states: Σ_s, P(s'|s,a).
- R — immediate reward; then γ × the *value of the next state*.
- That next-state value is itself Σ_a, π(a'|s') Q^{π}(s',a') — Q averaged over the next action **as chosen by π**.

Intuition: same recursion as for V, but anchored at a state-action pair. The next action a' is averaged under π — that "follow π afterwards" averaging is exactly what makes this an **expectation** (on-policy) equation rather than an optimality one.

Bellman *optimality* equations (for the optimal policy):

$$V^*(s) = \max_a \sum_{s'} P(s' \mid s, a)\big[\, R + \gamma V^*(s') \,\big]$$

Reading it term by term.

- The only change from the expectation equation: the action average Σ_a π(a|s) is replaced by **$\max_a$** — instead of averaging over what π does, take the *best* action.
- Σ_s, P(s'|s,a)[R + γV^*(s')] — the value of committing to action a (expected reward + discounted optimal value next).

Intuition: the optimal value of a state equals the value of its single best action. The **max** is what makes this equation **nonlinear**, so it has no direct linear solve — it is solved by iteration (value/policy iteration).

$$Q^*(s, a) = \sum_{s'} P(s' \mid s, a)\big[\, R + \gamma \max_{a'} Q^*(s', a') \,\big]$$

Reading it term by term.

- a is fixed, so average over next states: Σ_s, P(s'|s,a).
- R + γ $\max_a$, Q*(s',a') — *take a now, then assume we act optimally* from the next state onward (that is the inner max).

Intuition: "do a now, behave optimally forever after." Because the max sits on the *next* action and the value is indexed by (s,a), you can act greedily with **no model** — just take arg $\max_a$ Q^*(s,a). This is precisely why control methods learn Q rather than V.

How many are there? Four canonical equations: expectation for V and Q, and optimality for V and Q. (Equivalently: two *kinds* — expectation and optimality — each written for V and Q.) The expectation equations are **linear** and solvable by linear algebra; the optimality equations

are **nonlinear** (because of the max) and solved by iteration (value/policy iteration). Each defines a contraction operator whose unique fixed point is the corresponding value function.

4. What is a policy iteration algorithm and what are its stages?

Policy iteration computes the optimal policy by alternating two stages until the policy stops changing:

1. **Policy Evaluation.** Given the current policy π, compute V^{π} by solving the Bellman expectation equation (either exactly via linear solve, or iteratively by repeatedly applying the Bellman expectation backup until convergence).
2. **Policy Improvement.** Make the policy greedy with respect to the freshly computed value:

$$\pi'(s) \leftarrow \arg\max_a \sum_{s'} P(s' \mid s,a)\big[\, R + \gamma V^{\pi}(s') \,\big]$$

Reading it term by term.

- For each state s, score every candidate action a by its **one-step lookahead** $\Sigma_{s'}$ P(s'|s,a)[R + γV^{π}(s')] — this quantity is exactly Q^{π}(s,a).
- arg $\max_a$ picks the highest-scoring action; the arrow ← assigns it as the new policy's choice in s.

Intuition: "look one step ahead using the freshly computed V^{π}, then act greedily." Note this lookahead **needs the model P**; if instead we had Q^{π}, we could pick arg $\max_a$ Q^{π}(s,a) model-free.

Repeat: $\pi \to V^{\pi} \to \pi' \to V^{\pi'} \to \dots$ Because each improvement step produces a policy that is no worse (policy improvement theorem) and there are finitely many deterministic policies, the process converges to π* in a finite number of iterations.

Related: *Value iteration* collapses the two stages — it does a single Bellman *optimality* backup per state per sweep instead of fully evaluating each policy. *Generalized Policy Iteration (GPI)* is the umbrella idea that almost all RL methods interleave (partial) evaluation and (partial) improvement.

5. What are the differences between model-free and model-based methods?

- **Model-based** methods *have or learn* the dynamics P and reward R, and use them to **plan** — e.g. dynamic programming, Dyna, MCTS, LQR, world models. They can simulate hypothetical experience without touching the real environment.

- **Model-free** methods learn a value function and/or policy **directly from experience**, without ever estimating P or R — e.g. Q-learning, SARSA, REINFORCE, PPO, SAC.

Trade-offs.

Aspect	Model-based	Model-free
Sample efficiency	High (reuses model)	Lower (needs many env steps)
Computation	High (planning)	Lower per step
Asymptotic performance	Limited by model errors	Often higher / more robust
Implementation	Harder (must learn/exploit model)	Simpler, more stable

Model errors can *compound* during multi-step planning ("model bias"), so model-based methods shine when data is expensive and dynamics are learnable.

6. What is the difference between prediction and control?

- **Prediction (policy evaluation):** given a **fixed** policy π, estimate its value function V^{π} / Q^{π}. The policy does not change; we only *forecast* how well it does.
- **Control (optimization):** find the **best** policy π^* (and V*/Q*). Here the policy is improved over time.

Prediction is a sub-routine of control: most control algorithms repeatedly predict the current policy's value and then improve the policy (GPI). For example, TD(0) is a prediction algorithm; SARSA and Q-learning are control algorithms built on top of TD prediction.

7. Describe the differences between Monte-Carlo and Temporal-Difference?

Both estimate value functions from sampled experience without a model, but they differ in *what target* they use to update.

- **Monte-Carlo (MC):** waits until the **end of an episode** and updates toward the **actual observed return** G_t:

$$V(S_t) \leftarrow V(S_t) + \alpha\big[\, G_t - V(S_t) \,\big]$$

Reading it term by term.

- $V(S_t)$ — current estimate for the visited state (the thing we update).
- G_t — the **actual return**, obtained by running the episode to the end and summing discounted rewards.

- [$G_t - V(S_t)$] — the **error**: how far the estimate was from what really happened.
- $\alpha \in (0,1]$ — **step size**; move a fraction α of the way toward G_t.
- $\leftarrow$ — "becomes": new estimate = old + α·error.

Intuition: once the true return is known, nudge the estimate toward it. The target G_t is a genuine sample of the return, so the update is **unbiased** — but you must wait for the episode to end and G_t is **noisy** (high variance).

- **Temporal-Difference (TD(0)):** updates after **one step** toward a **bootstrapped** target that uses the current estimate of the next state:

$$V(S_t) \leftarrow V(S_t) + \alpha\big[\, R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \,\big]$$

Reading it term by term.

- $R_{t+1} + \gamma V(S_{t+1})$ — the **target**: one real reward plus our *current estimate* of the next state's value. Using an estimate inside the target is called **bootstrapping**.
- [target $- V(S_t)$] — the **TD error** δ_t: the surprise between the old prediction and this one-step-improved guess.
- α — step size; $\leftarrow$ — "becomes".

Intuition: don't wait for G_t — take a single step and trust the existing estimate for everything after. This updates **online** with **low variance**, but it is **biased** while V is still inaccurate (the target leans on a wrong estimate).

The bracket is the **TD error** δ_t.

Property	Monte-Carlo	TD
Bootstrapping	No	Yes
Bias	Unbiased	Biased (uses estimates)
Variance	High	Low
Needs episodes to end	Yes	No (online, continuing tasks)
Markov assumption	Not required	Exploited
Convergence target	true V^π	true V^π (under conditions)

MC is unbiased but high-variance and only works for episodic tasks; TD is biased but low-variance, learns online, and is usually more data-efficient. They are the two endpoints of the spectrum unified by TD(λ).

8. What is TD(k) and TD(λ)?

n-step TD (TD(k)). Instead of bootstrapping after 1 step (TD(0)) or never (MC), use the *n-step return*: sum n real rewards, then bootstrap:

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

Reading it term by term.

- First n terms ($R_{t+1} \dots \gamma^{n-1} R_{t+n}$) — the *actual* discounted rewards collected over n real steps.
- $\gamma^n V(S_{t+n})$ — the **bootstrap**: after n real rewards, fall back on the value estimate for everything beyond.

Intuition: a dial between TD and MC. n = 1 gives TD(0) (one reward + bootstrap); n = ∞ gives Monte-Carlo (all real rewards, no bootstrap). Larger n means **less bias** (more real data in the target) but **more variance**.

n = 1 gives TD(0); n = ∞ gives Monte-Carlo. n controls the **bias–variance trade-off**: larger n → less bias, more variance.

TD(λ). Rather than commit to a single n, TD(λ) takes a *geometrically weighted average* of all n-step returns — the **λ-return**:

$$G_t^{\lambda} = (1-\lambda) \sum_{n \geq 1} \lambda^{n-1} G_t^{(n)}, \qquad \lambda \in [0, 1]$$

Reading it term by term.

- Instead of picking one n, **blend all** n-step returns $G_t^{(n)}$.
- λ^{n-1} — the weight on the n-step return; longer horizons receive geometrically smaller weight.
- $(1-\lambda)$ — a normalizer so the weights sum to 1 (a proper weighted average).

Intuition: a single number λ controls the whole TD↔MC spectrum. λ = 0 puts all weight on the 1-step return (⇒ TD(0)); λ = 1 puts all weight on the full return (⇒ Monte-Carlo); in between it smoothly averages every horizon.

λ = 0 reduces to TD(0); λ = 1 reduces to Monte-Carlo. TD(λ) is implemented online and efficiently using **eligibility traces**: a trace e(s) marks how recently/frequently each state was visited, and every state is updated by $\delta_t \cdot e(s)$:

$$e_t(s) = \gamma\lambda\, e_{t-1}(s) + \mathbf{1}[S_t = s], \qquad V(s) \leftarrow V(s) + \alpha\, \delta_t\, e_t(s)$$

Reading it term by term.

- $e_t(s)$ — the **eligibility trace** of state s: a short-term memory of how recently and how often s was visited.
- $\gamma\lambda\, e_{t-1}(s)$ — **decay** the previous trace by $\gamma\lambda$ each step, so older visits fade.
- $\mathbf{1}[S_t = s]$ — add 1 whenever s is the state just visited (a "bump").
- $V(s) \leftarrow V(s) + \alpha\, \delta_t\, e_t(s)$ — apply the current TD error δ_t to **every** state, scaled by its trace.

Intuition: one TD error is spread **backward** over all recently-visited states, each credited in proportion to its trace. This realizes the λ-return **online**, in a single pass, without waiting for the episode to finish.

This gives a smooth interpolation between TD and MC and propagates credit backward over many steps in a single online pass.

9. Describe the SARSA algorithm.

SARSA is an **on-policy, model-free TD control** algorithm that learns Q^π for the policy it is actually following. Its name comes from the tuple it uses: (S, A, R, S′, A′).

Update rule:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[\, R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\,\big]$$

Reading it term by term.

- Same shape as the TD(0) update, but for **action-values** Q(S,A).
- Target $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$ uses A_{t+1}, the action **actually taken next** by the behavior policy — the final "A" in the tuple (S, A, R, S′, A′).
- The bracket is the TD error; α is the step size.

Intuition: the target evaluates the policy the agent is **really following**, exploratory moves included — that is what makes SARSA **on-policy**. Because the cost of exploration shows up in the values, it learns cautious, "safe" behavior.

Pseudocode:

1. Initialize Q arbitrarily.
2. For each episode: choose A from S using a policy derived from Q (e.g. ε-greedy).
3. For each step: take A, observe R, S′; choose A′ from S′ using the same ε-greedy policy.
4. Update Q(S,A) with the rule above; set $S \leftarrow S'$, $A \leftarrow A'$.

Because the bootstrap target uses the **actually chosen** next action A′ (drawn from the behavior policy), SARSA evaluates and improves the *same* policy. It is therefore **on-policy** and

tends to learn safer behavior in risky environments (the classic "cliff walking" example: SARSA learns a safe path away from the cliff because exploration costs are reflected in its values).

10. Describe the Q-learning algorithm.

Q-learning is an **off-policy, model-free TD control** algorithm that learns the optimal action-value function Q* directly, regardless of the policy used to generate data.

Update rule:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[\, R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \,\big]$$

Reading it term by term.

- Identical to SARSA **except** the next-action term is $\max_a$, $Q(S_{t+1}, a')$ instead of $Q(S_{t+1}, A_{t+1})$.
- $\max_a$, — assume the **greedy** (best) next action, no matter what the behavior policy actually did.

Intuition: the target describes the **optimal** policy even though the data may come from an exploratory one — that is what makes Q-learning **off-policy**. It learns Q^* directly and ignores exploratory mistakes, so it prefers the "risky but optimal" path that SARSA shies away from.

The key difference from SARSA is the **max** in the target: the bootstrap uses the *greedy* next action, not the action actually taken. So the behavior policy (e.g. ε-greedy, used to explore) can differ from the target policy (greedy) being learned — hence *off-policy*.

Pseudocode:

1. Initialize Q.
2. For each step: choose A from S via ε-greedy on Q; take A, observe R, S′.
3. Update with the rule above; S ← S′.

Under standard conditions (every state-action visited infinitely often, suitable step sizes) Q-learning converges to Q*. In the cliff-walking example it learns the *optimal* (risky, short) path because its target ignores exploratory mistakes. Q-learning is the foundation of DQN.

11. What is the difference between on-policy and off-policy algorithms?

- **On-policy:** the algorithm learns about and improves the **same** policy that generates the data (the *behavior* = *target* policy). It must keep exploring with the policy it evaluates. Examples: SARSA, REINFORCE, A3C, PPO, TRPO.

- **Off-policy:** the algorithm learns a **target** policy that is different from the **behavior** policy used to collect data. This decouples exploration from the policy being optimized and enables **data reuse** (replay buffers). Examples: Q-learning, DQN, DDPG, TD3, SAC.

Why it matters. Off-policy learning is more sample-efficient (can reuse old data, learn from demonstrations or other agents) but is harder to stabilize, especially with function approximation (part of the “deadly triad”); it often needs *importance sampling* or special tricks. On-policy learning is more stable and has lower-variance gradient estimates but is data-hungry because it must discard data after each policy update.

▲ Function approximation

1. Describe model-free prediction using function approximators and Monte-Carlo and Temporal Difference.

When state spaces are large or continuous we cannot store a table of values, so we approximate $V^{\pi}(s) \approx \hat{v}(s; w)$ with parameters w (linear features or a neural network). We fit w by gradient descent on the squared error between the estimate and a **target**.

The general update is:

$$w \leftarrow w + \alpha \left[\text{target} - \hat{v}(S_t; w) \right] \nabla_w \hat{v}(S_t; w)$$

Reading it term by term.

- w — the **parameters** of the approximator (linear feature weights, or all the weights of a neural net).
- $\hat{v}(S_t; w)$ — the approximator's **predicted** value for S_t.
- target — the value we wish the prediction matched (G_t for MC, $R_{t+1}+\gamma\hat{v}(S_{t+1})$ for TD).
- [target − $\hat{v}$] — the prediction **error**.
- $\nabla_w \hat{v}(S_t; w)$ — the **gradient**: the direction in weight space that most increases the prediction at S_t.
- α — step size.

Intuition: nudge the weights so $\hat{v}(S_t)$ moves toward the target — "error × which way to push each weight." The tabular update is the special case where the gradient is 1 for the visited state and 0 everywhere else.

- **MC with function approximation:** target = the actual return G_t. This is a true stochastic-gradient step on $\mathbb{E}[(G - \hat{v})^2]$; it is **unbiased** and converges (to a local optimum for nonlinear, to the global LS solution for linear), but high variance and needs complete episodes.
- **TD(0) with function approximation:** target = $R_{t+1} + \gamma\, \hat{v}(S_{t+1}; w)$. Because the target itself depends on w, this is a **semi-gradient** method (we do *not* differentiate through the target). It is biased and lower-variance, learns online, and for **linear** approximation converges to the *TD fixed point* (close to but not exactly the minimum MSE). With **nonlinear** approximators and off-policy data it can diverge.

The same applies to action-values $\hat{q}(s,a;w)$ for control, and to TD(λ) with eligibility traces on the parameters.

2. Describe potential problems when using function approximators in RL. What is the deadly triad?

Combining approximation with RL introduces instabilities not present in tabular methods:

- **Bootstrapping bias:** TD targets depend on current (wrong) estimates; generalization spreads errors to other states.
- **Non-stationary, correlated data:** the data distribution shifts as the policy changes, and consecutive samples are highly correlated, violating i.i.d. assumptions of SGD.
- **Moving targets:** the regression target changes as parameters update.
- **Overestimation (maximization bias):** the max operator over noisy estimates biases values upward — not unique to function approximation, but amplified by it.

The deadly triad (Sutton & Barto) — instability/divergence is likely when **all three** of the following are combined:

1. **Function approximation** (especially nonlinear, e.g. neural nets),
2. **Bootstrapping** (TD-style targets that use current estimates),
3. **Off-policy training** (learning about a policy different from the one generating data).

Any two are usually safe; all three together can cause value estimates to blow up. DQN's tricks (experience replay, target networks) are precisely engineering remedies that make the triad workable in practice.

3. Describe the LSMC and LSTD algorithms.

For **linear** value approximation $\hat{v}(s) = \varphi(s)^T w$, we can solve for w in closed form by least squares instead of incremental SGD — far more sample-efficient.

- **LSMC (Least-Squares Monte-Carlo):** minimize $\Sigma_t(G_t - \varphi(S_t)^T w)^2$. The solution is ordinary least squares:

$$w = \left(\sum_t \phi(S_t)\, \phi(S_t)^\top \right)^{-1} \sum_t \phi(S_t)\, G_t$$

Reading it term by term.

- $\varphi(S_t)$ — the **feature vector** of state S_t (length d); $\hat{v}(s) = \varphi(s)^\top w$.
- $\varphi(S_t)\varphi(S_t)^\top$ — an outer product (a d×d matrix); summed over the data it is the (unnormalized) **feature covariance**.

- $\Sigma_t \varphi(S_t)\, G_t$ — features correlated with the observed returns.
- $(\Sigma\, \varphi\varphi^\top)^{-1}(\Sigma\, \varphi\, G_t)$ — the **ordinary least-squares** solution.

Intuition: this is just linear regression of returns G_t on features φ, solved in **closed form** — no learning rate, uses the whole batch at once.

Uses the unbiased MC return as target.

- **LSTD (Least-Squares Temporal-Difference):** set the expected TD update to zero. The TD fixed point is:

$$w = A^{-1}b, \qquad A = \sum_t \phi(S_t)\big(\phi(S_t) - \gamma\phi(S_{t+1})\big)^\top, \qquad b = \sum_t \phi(S_t)\, R_{t+1}$$

Reading it term by term.

- $\varphi(S_t) - \gamma\varphi(S_{t+1})$ — the **feature temporal difference** (current feature minus discounted next feature).
- $A = \Sigma\, \varphi(S_t)(\varphi(S_t) - \gamma\varphi(S_{t+1}))^\top$ — a d×d matrix pairing each feature with that TD feature.
- $b = \Sigma\, \varphi(S_t)\, R_{t+1}$ — features paired with immediate rewards.
- $w = A^{-1}b$ — the **TD fixed point**, in closed form.

Intuition: the batch version of TD(0). Rather than regressing on full returns (LSMC), it solves directly for the weights at which the average TD error, projected onto the features, is **zero**. No step size; cost is $O(d^2)$–$O(d^3)$ from forming and inverting A.

This directly computes the TD solution from a batch of data, no step-size tuning, using all samples efficiently.

LSTDQ / LSPI: the action-value version (LSTDQ) plugged into policy iteration gives **Least-Squares Policy Iteration (LSPI)** — a sample-efficient off-policy batch control method. Strengths: data-efficient, no learning-rate tuning. Weaknesses: $O(d^2)$–$O(d^3)$ cost in the number of features d, and restricted to linear approximation.

4. Describe the DQN algorithm. What problems of using function approximators in RL DQN aims to solve?

Deep Q-Network (DQN) (Mnih et al., 2015) scales Q-learning to high-dimensional inputs (Atari from raw pixels) using a deep CNN Q(s, a; θ) and learns by minimizing the TD error:

$$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim D}\Big[\big(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\big)^2\Big]$$

Reading it term by term.

- θ — weights of the **online** Q-network being trained; θ^- — weights of a periodically-frozen copy, the **target network**.
- $(s,a,r,s') \sim D$ — sample a past transition from the **replay buffer** D.
- $r + \gamma \max_{a'} Q(s',a'; \theta^-)$ — the Q-learning **target**, computed with the *frozen* θ^- so it stays still while we fit.
- $Q(s,a; \theta)$ — the current **prediction**.
- $(\text{target} - \text{prediction})^2$ — squared TD error; $\mathbb{E}[\cdot]$ averages it over the minibatch.

Intuition: plain regression of $Q(s,a;\theta)$ onto a Q-learning target, with the two stabilizers built in — **replay** (the $\sim D$ sampling decorrelates data) and the **target network** (θ^- stops the target from chasing its own tail). Together they tame the deadly triad.

Naively combining Q-learning with a neural net hits the **deadly triad** and diverges. DQN introduces two stabilizing tricks:

1. **Experience replay (buffer D).** Store transitions and sample random minibatches for updates. This *breaks temporal correlation* between consecutive samples (more i.i.d.-like data) and *reuses* each transition many times (sample efficiency). Addresses the correlated/non-stationary-data problem.
2. **Target network (θ^-).** Compute the bootstrap target with a *separate*, periodically-copied (frozen) network θ^-, updated every C steps. This makes the regression target *stationary* over short horizons, addressing the moving-target problem and preventing oscillation/divergence.

Additional engineering: reward clipping, frame stacking, ε-greedy exploration with annealing.

Problems solved: instability and divergence of deep off-policy Q-learning, specifically (a) correlated data, (b) non-stationary targets. Later extensions fix remaining issues — **Double DQN** (overestimation), **Dueling DQN** (V/A decomposition), **Prioritized Replay** (sample important transitions), **Distributional/C51**, **Noisy Nets** — combined in **Rainbow**.

✦ Policy-gradient methods

1. Why/when use policy methods and value methods?

Value-based methods learn Q and act greedily; **policy-based** methods parametrize and optimize the policy π_θ directly.

Use policy methods when:

- **Continuous or high-dimensional action spaces** — argmax_a Q(s,a) is intractable, but a parametrized policy outputs actions directly.
- **Stochastic optimal policies are needed** — e.g. partially observed games (rock-paper-scissors), where the best policy is randomized; value-greedy policies are deterministic.
- **Smooth/stable improvement** — small parameter changes give small policy changes; value methods can change the greedy action abruptly.
- You want to **inject priors** on the policy form, or need good **convergence guarantees** (PG converges to a local optimum).

Use value methods when:

- **Discrete, small action spaces** where argmax is easy.
- **Sample efficiency** matters and off-policy replay is desirable (DQN reuses data; vanilla PG is on-policy and data-hungry).

Drawbacks of PG: high-variance gradient estimates, sample inefficiency, and convergence only to local optima. **Actor-critic** methods combine both: a policy (actor) plus a value function (critic) to reduce variance.

2. List two gradient-free policy algorithms?

Gradient-free (black-box) methods optimize policy parameters without computing $\nabla\theta$ via backprop through the return:

1. **Evolutionary Strategies (ES)** — e.g. CMA-ES, OpenAI's ES: perturb parameters with Gaussian noise, evaluate fitness (episode return), and move toward higher-return perturbations.
2. **Cross-Entropy Method (CEM)** — sample parameter vectors from a distribution, keep the top-performing "elite" fraction, refit the distribution to them, repeat.

(Other examples: genetic algorithms, simulated annealing, random/hill-climbing search, Augmented Random Search (ARS).)

3. Evolutionary Strategies, what are their strengths and weaknesses?

ES treats the return as a black-box function F(θ) of parameters and estimates a gradient by sampling perturbations:

$$\nabla_\theta \, \mathbb{E}_{\varepsilon \sim N(0,I)} \left[F(\theta + \sigma\varepsilon) \right] \approx \frac{1}{\sigma} \, \mathbb{E}_\varepsilon \left[F(\theta + \sigma\varepsilon) \, \varepsilon \right]$$

Reading it term by term.

- F(θ) — the black-box **fitness**: the total episode return of the policy with parameters θ (no gradients needed inside).
- ε ∼ N(0,I) — a random Gaussian **perturbation** vector.
- θ + σε — the parameters jittered by noise of scale σ.
- LHS — the gradient of the **smoothed** objective (return averaged over jitters).
- RHS $(1/\sigma)\ \mathbb{E}_\varepsilon[\ F(\theta+\sigma\varepsilon)\ \varepsilon\ ]$ — estimate that gradient by sampling many perturbations and taking a **return-weighted average of the noise directions**.

Intuition: perturbations that *raised* the return pull θ toward them; those that *lowered* it push θ away. Only scalar returns and the random seeds need to be shared between workers — no backprop through policy or environment — which is why ES scales to thousands of CPUs.

The population evaluates many perturbed policies in parallel and θ moves in the fitness-weighted direction.

Strengths.

- **Massively parallel / scalable** — workers only need to share scalar returns and random seeds; near-linear scaling to thousands of CPUs (OpenAI ES solved MuJoCo/Atari in minutes of wall-clock time).
- **No backpropagation, no value function, no gradients** — works with non-differentiable rewards and long horizons.
- **Robust to sparse/delayed rewards and long credit assignment** — it only cares about total episode return; immune to vanishing gradients.
- Few hyperparameters; tolerant of noisy, partially observed environments; good exploration in parameter space.

Weaknesses.

- **Sample-inefficient** in terms of environment interactions (needs many full episode rollouts), though wall-clock-efficient with enough compute.
- Scales poorly with the number of parameters in *sample complexity* (estimating a gradient by random search in high dimensions is statistically weak).

- Ignores the temporal structure / per-step information that policy-gradient and TD methods exploit.

4. Compare on-policy methods and off-policy methods. List a few algorithms in each class.

- **On-policy:** the target policy = behavior policy. Must collect fresh data with the current policy after every update; cannot reuse old data without correction. *Pros:* stable, low-bias gradient estimates. *Cons:* sample inefficient. **Examples:** SARSA, REINFORCE, A2C/A3C, TRPO, PPO, IMPALA (with V-trace correction).
- **Off-policy:** target policy ≠ behavior policy. Can reuse stored experience (replay buffers), learn from demonstrations/other agents. *Pros:* sample-efficient, decouples exploration. *Cons:* harder to stabilize (deadly triad), may need importance sampling. **Examples:** Q-learning, DQN (+ Rainbow), DDPG, TD3, SAC, Retrace, ACER.

Actor-critic methods exist in both camps: A3C/PPO are on-policy; SAC/DDPG are off-policy. IMPALA is "almost on-policy" — it uses V-trace importance weights to correct for the slight off-policyness of distributed actors.

5. Describe policy gradient theorem (show the proof - for ambitious students).

Goal. Maximize $J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[\,R(\tau)\,]$, the expected return of trajectories τ generated by π_θ.

Theorem.

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, Q^\pi(s_t, a_t)\Big]$$

Reading it term by term.

- $J(\theta)$ — the expected return of policy π_θ; $\nabla_\theta J$ is the direction to step θ for **gradient ascent**.
- $\nabla_\theta \log \pi_\theta(a_t|s_t)$ — the **score**: the direction in θ that makes action a_t *more probable* in s_t.
- $Q^\pi(s_t,a_t)$ — how good that action actually was.
- $\mathbb{E}_{\pi_\theta}[\,\Sigma_t \cdot\,]$ — average over trajectories the policy itself generates.

Intuition: "raise the log-probability of each action, weighted by how good it was." High-Q actions get their probability pushed up, low-Q actions pushed down. Because Q simply *multiplies* the score, the environment is never differentiated.

Proof (trajectory / likelihood-ratio form). The probability of a trajectory $\tau = (s_0,a_0,s_1,\ldots)$ under π_θ is

$$p_\theta(\tau) = \rho(s_0) \prod_t \pi_\theta(a_t \mid s_t)\, P(s_{t+1} \mid s_t, a_t)$$

Reading it term by term.

- τ — a whole **trajectory** (s_0, a_0, s_1, a_1, …).
- $\rho(s_0)$ — probability of the starting state.
- $\Pi_t\, \pi_\theta(a_t|s_t)$ — probability the **policy** chose each action.
- $\Pi_t\, P(s_{t+1}|s_t,a_t)$ — probability the **environment** produced each next state.

Intuition: a trajectory occurs only if the start, every action, and every transition occur — so multiply their probabilities. Note **only the π_θ factors depend on θ**; ρ and P do not — the hinge of the whole proof.

Take logs: $\log p_\theta(\tau) = \log \rho(s_0) + \Sigma_t[\, \log \pi_\theta(a_t \mid s_t) + \log P(s_{t+1} \mid s_t,a_t)\,]$. The dynamics ρ and P do **not** depend on θ, so

$$\nabla_\theta \log p_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

Reading it term by term.

- Taking the log of the product above turns it into a **sum** of logs.
- Differentiating that sum in θ: the $\log \rho(s_0)$ and the $\log P(s_{t+1}|s_t,a_t)$ terms contain no θ, so they **vanish**.
- Only $\Sigma_t\, \nabla_\theta \log \pi_\theta(a_t|s_t)$ survives.

Intuition: the unknown dynamics drop out under the gradient. This is exactly why policy gradients are **model-free** — you never need to know, or differentiate, the transition model P.

Now use the **log-derivative (REINFORCE) trick** $\nabla p = p\, \nabla \log p$:

$$\nabla_\theta J = \int p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau)\, R(\tau)\, d\tau = \mathbb{E}_\tau\Big[R(\tau) \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big]$$

Reading it term by term.

- Start from $J(\theta) = \int p_\theta(\tau)\, R(\tau)\, d\tau$ and apply the **log-derivative trick** $\nabla p = p\, \nabla \log p$, which slips the gradient inside as a factor $\nabla \log p$.
- $\int p_\theta(\tau)(\,\cdot\,)\, d\tau$ is, by definition, an expectation $\mathbb{E}_\tau[\,\cdot\,]$.

- Substitute $\nabla_\theta \log p_\theta(\tau) = \Sigma_t \nabla_\theta \log \pi_\theta$ from the previous line.

Intuition: the gradient of an expectation has become an expectation we can **sample**: run the policy, and weight each trajectory's summed scores by its total return $R(\tau)$. That sampleable form is precisely REINFORCE.

Crucially, **the model-free property** appears: the gradient needs no knowledge of P. Rewards earned *before* t cannot be affected by a_t, so they vanish in expectation; for the future rewards the tower rule gives $\mathbb{E}[\, \Sigma_{k\geq t} \gamma^{k-t} R_{k+1} \mid s_t, a_t \,] = Q^\pi(s_t, a_t)$, yielding the Q^π form above.

(Strictly, the discounted objective also carries a γ^t weighting on the state distribution, almost universally dropped in practice — see Thomas, 2014.) Subtracting any state-dependent **baseline** b(s) leaves the gradient unbiased (because $\mathbb{E}[\nabla \log \pi \cdot b(s)] = 0$) but reduces variance; choosing $b(s) = V^\pi(s)$ gives the **advantage** form:

$$\nabla_\theta J = \mathbb{E}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, A^\pi(s_t, a_t)\Big], \qquad A^\pi = Q^\pi - V^\pi$$

Reading it term by term.

- Same as the theorem, but Q^π is replaced by the **advantage** $A^\pi = Q^\pi - V^\pi$.
- $V^\pi(s)$ acts as the **baseline** b(s); $A^\pi(s,a)$ measures how much better action a is than the state's *average* action.

Intuition: subtracting the baseline V^π leaves the gradient **unbiased** (the baseline term averages to zero) but sharply **reduces variance** — we now push on *relative* quality. $A > 0 \Rightarrow$ raise the action's probability; $A < 0 \Rightarrow$ lower it; $A \approx 0 \Rightarrow$ leave it alone.

6. What is the Reinforce algorithm?

REINFORCE (Williams, 1992) is the simplest Monte-Carlo policy-gradient algorithm. It uses complete-episode returns as an unbiased estimate of Q^π in the policy-gradient theorem.

Algorithm:

1. Run an episode with π_θ, collecting (s_t, a_t, r_t).
2. Compute returns $G_t = \Sigma_{k\geq t} \gamma^{k-t} r_{k+1}$.
3. Update parameters by gradient *ascent*:

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \big(G_t - b(s_t)\big)$$

Reading it term by term.

- After a full episode, for each step take the **score** $\nabla_\theta \log \pi_\theta(a_t|s_t)$.
- G_t — the actual return from t, the Monte-Carlo estimate of $Q^\pi(s_t,a_t)$.
- $b(s_t)$ — a **baseline** (typically a learned $\hat{V}(s_t)$) subtracted to cut variance.
- α — step size; the + sign makes this gradient *ascent* (we maximize return).

Intuition: the practical, sampled form of the theorem — weight each action's score by how much its observed return **beat the baseline**. Unbiased, but high-variance because G_t is a noisy whole-episode sample.

The optional **baseline** b(s) (typically a learned $\hat{V}(s)$) does not bias the gradient but greatly reduces variance. REINFORCE is on-policy, unbiased, but **high-variance** and sample-inefficient (must wait for episode ends). Actor- critic methods replace G_t with a bootstrapped/ critic estimate to cut variance.

7. Describe the A3C algorithm. What are its strengths and weaknesses? (we may ask about the pseudo-code).

A3C (Asynchronous Advantage Actor-Critic) (Mnih et al., 2016) is an on-policy actor-critic method that parallelizes data collection across many CPU workers *instead of* using a replay buffer.

- Multiple **worker** threads each have a copy of the policy π_θ (actor) and value $V(s; \theta_v)$ (critic) and interact with their own environment instance.
- Each worker runs n-step rollouts, computes the **advantage** $\hat{A}_t = \Sigma\, \gamma^i\, r_{t+i} + \gamma^n V(s_{t+n}) - V(s_t)$, and the gradients:

$$\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \hat{A}_t + \beta\, \nabla_\theta H\big(\pi_\theta(\cdot \mid s_t)\big)$$

Reading it term by term.

- $\nabla_\theta \log \pi_\theta(a_t|s_t)\, \hat{A}_t$ — the policy-gradient term, with an **estimated advantage** $\hat{A}_t$ from the n-step rollout and the critic.
- $H(\pi_\theta(\cdot|s_t))$ — the **entropy** of the policy at s_t (how spread-out the action distribution is).
- β — a small coefficient on the entropy bonus.
- $\nabla_\theta H$ — pushes the policy toward being *more* random.

Intuition: the first term improves the policy; the second keeps it from collapsing to a single deterministic action too soon, preserving exploration. β sets how strongly exploration is encouraged.

plus a value loss $(\hat{A}_t)^2$ for the critic. An **entropy bonus** H encourages exploration. - Workers **asynchronously** push gradients to (and pull params from) a shared global network — "Hogwild!"-style lock-free updates.

Strengths. No replay buffer (lower memory); decorrelation of data comes from *parallel diverse actors* rather than replay, so it works with **on-policy** actor-critic; CPU-friendly and fast in wall-clock time; entropy regularization aids exploration; reasonably robust, though less stable than synchronous A2C.

Weaknesses. Asynchrony causes **stale gradients** (workers update with slightly old parameters); sample-inefficient (on-policy, data discarded after use); sensitive to hyperparameters; the synchronous variant **A2C** was later shown to match or beat A3C with simpler, GPU-efficient batched updates.

8. What is IMPALA?

IMPALA (Importance-Weighted Actor-Learner Architecture) (Espeholt et al., 2018) is a highly **scalable distributed actor-critic** for training on many tasks at once.

Architecture: many **actors** generate trajectories and send them to one (or a few) central **learner(s)** that perform batched GPU updates. Unlike A3C, actors send **trajectories of experience** (not gradients), giving high throughput.

Because actors run a slightly **stale** policy μ (the policy lags behind the learner's π), the data is mildly **off-policy**. IMPALA corrects this with **V-trace**, an off-policy importance-sampled value target with *truncated* importance weights for stability:

$$v_s = V(s_s) + \sum_{t \geq s} \gamma^{t-s} \left(\prod_{i=s}^{t-1} c_i \right) \delta_t^V, \qquad \delta_t^V = \rho_t \big(r_t + \gamma V(s_{t+1}) - V(s_t) \big)$$

Reading it term by term.

- v_s — the corrected value **target** for state s (what the critic regresses toward); $V(s_s)$ is the current estimate it starts from.
- δ_t^V — a one-step TD error, but scaled by ρ_t.
- $\rho_t = \min(\bar{\rho}, \pi/\mu)$ — a **truncated importance weight**: how much more (or less) likely the action was under the *learner* π than under the *stale actor* μ, capped at $\bar{\rho}$.
- $\Pi_{i=s}^{t-1} c_i$, with $c_i = \min(\bar{c}, \pi/\mu)$ — a product of truncated weights that discounts the influence of TD errors further into the future.
- γ^{t-s} — the ordinary discount over the horizon.

Intuition: an **off-policy-corrected n-step target**. The importance weights fix the mismatch between the data-collecting policy μ and the learner π; truncating them ($\bar{\rho}$, $\bar{c}$) stops them from

exploding, trading a little bias for much lower variance. **$\bar{\rho}$ controls *which* value function you converge to; $\bar{c}$ controls *how fast*.**

where $\rho_t = \min(\bar{\rho}, \pi/\mu)$, $c_i = \min(\bar{c}, \pi/\mu)$. Truncation controls variance ($\bar{\rho}$ sets the fixed point, $\bar{c}$ controls convergence speed).

Strengths: very high throughput and scalability (decoupled actors/learner), data-efficient relative to A3C, principled off-policy correction, strong multi-task performance (DMLab-30). **Weaknesses:** V-trace truncation introduces bias; engineering complexity.

9. Derive the TRPO algorithm. What are its strengths and weaknesses? (we may ask about the pseudo-code).

TRPO (Trust Region Policy Optimization) (Schulman et al., 2015) is an on-policy method that takes the largest policy-improvement step that stays within a "trust region" so the policy does not change too much and collapse.

Derivation. The performance of a new policy $\tilde{\pi}$ relative to old π satisfies the identity:

$$J(\tilde{\pi}) = J(\pi) + \mathbb{E}_{\tau\sim\tilde{\pi}}\Big[\sum_t \gamma^t A^{\pi}(s_t, a_t)\Big]$$

Reading it term by term.

- $J(\tilde{\pi}) - J(\pi)$ — how much better the **new** policy $\tilde{\pi}$ is than the old π.
- $\mathbb{E}_{\tau\sim\tilde{\pi}}[\,\Sigma_t\, \gamma^t\, A^{\pi}(s_t,a_t)\,]$ — the expected discounted **advantage of the *old* policy**, but evaluated along trajectories generated by the *new* policy.

Intuition: a new policy improves exactly to the extent that, on the states it visits, it favors actions the old value function rated above average ($A^{\pi} > 0$). The snag: the expectation is under $\tilde{\pi}$, which we cannot sample until we already have it — which is why the next step introduces a surrogate.

The expectation under $\tilde{\pi}$ is unavailable, so we use the **surrogate** that samples states from the *old* policy and reweights actions by an importance ratio:

$$L_{\pi}(\tilde{\pi}) = \mathbb{E}_{s\sim\rho_{\pi},\, a\sim\pi}\Big[\frac{\tilde{\pi}(a \mid s)}{\pi(a \mid s)}\, A^{\pi}(s,a)\Big]$$

Reading it term by term.

- $s \sim \rho_{\pi}$ — sample states from the **old** policy's visitation (data we already have); $a \sim \pi$ — actions from the old policy too.
- $\tilde{\pi}(a|s)/\pi(a|s)$ — the **importance ratio**: reweight each old action by how much *more* likely the new policy makes it.

- $A^{\pi}(s,a)$ — the old advantage.

Intuition: we cannot sample under $\tilde{\pi}$ (previous formula), so we approximate that expectation by **reweighting old data** with the probability ratio. This is a faithful surrogate only while $\tilde{\pi}$ stays close to π — hence the trust-region constraint that follows.

A bound (Kakade & Langford; Schulman) shows $J(\tilde{\pi}) \geq L_{\pi}(\tilde{\pi}) - C \cdot \max_s D_{KL}(\pi \| \tilde{\pi})$. Maximizing this lower bound guarantees monotonic improvement, but the penalty makes steps tiny. TRPO instead enforces a **hard KL constraint**:

$$\max_{\theta} \ \mathbb{E}\left[\frac{\pi_\theta}{\pi_{\text{old}}} \hat{A}\right] \quad \text{s.t.} \quad \mathbb{E}\left[D_{\text{KL}}(\pi_{\text{old}} \,\|\, \pi_\theta)\right] \leq \delta$$

Reading it term by term.

- $\max_{\theta} \mathbb{E}[\ (\pi_\theta/\pi_{\text{old}})\ \hat{A}\ ]$ — **maximize the surrogate**: raise the probability of high-advantage actions.
- "s.t." — *subject to* the following constraint.
- $\mathbb{E}[\ D_{KL}(\pi_{\text{old}} \,\|\, \pi_\theta)\ ] \leq \delta$ — keep the **KL divergence** (how much the action distribution shifted) below a trust-region radius δ.

Intuition: take the largest improvement step you can **without** moving the policy more than δ in distribution — the constraint blocks the over-large updates that make naive policy gradients collapse. Solved by a natural-gradient direction (conjugate gradient on the Fisher matrix) followed by a backtracking line search.

This is solved by a **natural-gradient** step: linearize the objective, take a quadratic (Fisher-matrix F) approximation of the KL, giving the search direction $F^{-1}g$ computed via **conjugate gradient** (avoiding explicit F^{-1}), then a **line search** (backtracking) to enforce the constraint and ensure actual improvement.

Strengths: near-monotonic improvement, stable, good for continuous control, robust to step-size choice. **Weaknesses:** complex to implement (CG + Fisher-vector products + line search), computationally expensive per update, hard to combine with parameter sharing/noise/architectures; superseded in practice by the simpler PPO.

10. Derive the PPO algorithm. What are its strengths and weaknesses? (we may ask about the pseudo-code).

PPO (Proximal Policy Optimization) (Schulman et al., 2017) keeps TRPO's trust-region idea but replaces the hard constraint with a simple, first-order objective optimizable by SGD/Adam.

Let $r_t(\theta) = \pi_\theta(a_t \mid s_t) \,/\, \pi_{\text{old}}(a_t \mid s_t)$ be the probability ratio. The **clipped surrogate objective**:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\,\hat{A}_t,\quad \text{clip}(r_t(\theta),\,1-\epsilon,\,1+\epsilon)\,\hat{A}_t\big)\Big]$$

Reading it term by term.

- $r_t(\theta) = \pi_\theta/\pi_{\text{old}}$ — the probability **ratio** (same object TRPO used).
- $r_t\,\hat{A}_t$ — the unclipped surrogate.
- $\text{clip}(r_t, 1-\varepsilon, 1+\varepsilon)$ — force the ratio into the band $[1-\varepsilon, 1+\varepsilon]$ so the policy cannot move "too far" on this sample.
- min(unclipped, clipped) — take the **smaller** (more pessimistic) of the two.

Intuition: once the policy has shifted far enough in a good direction (ratio past $1\pm\varepsilon$) the clip **flattens** the objective, removing any incentive to push further. The **min** ensures clipping only removes the incentive to over-step — it never rewards moving away. This reproduces TRPO's trust region with a plain first-order objective (no second-order math).

Intuition / derivation. We want to increase probability of good actions ($\hat{A}>0$) and decrease bad ones ($\hat{A}<0$), but not move too far from π_{old}. The clip caps the ratio to $[1-\varepsilon, 1+\varepsilon]$ so that once the policy has moved "enough" in a beneficial direction the incentive flattens (gradient becomes zero); the **min** ensures the objective is a *pessimistic lower bound* — clipping only removes incentive to over-step, it never rewards moving away. This achieves TRPO's effect without second-order computation.

Full PPO loss (actor-critic):

$$L = \mathbb{E}_t\Big[L^{\text{CLIP}} - c_1\big(V_\theta(s_t) - V_t^{\text{targ}}\big)^2 + c_2\,H\big(\pi_\theta(\cdot \mid s_t)\big)\Big]$$

Reading it term by term.

- L^{CLIP} — the clipped policy objective above (maximized).
- $-c_1(V_\theta(s_t) - V_t^{\text{targ}})^2$ — the **critic loss**: squared error fitting the value function to its target (minus sign because the whole L is *maximized* but this error must shrink).
- $+c_2\,H(\pi_\theta(\cdot|s_t))$ — an **entropy bonus** for exploration.
- c_1, c_2 — coefficients balancing the three pieces.

Intuition: one combined loss trains actor, critic, and exploration together — improve the policy, fit the value function, and stay a little random.

Advantages are usually estimated with **GAE**. PPO runs several epochs of minibatch SGD over each batch of on-policy data, then refreshes the data. (An alternative *adaptive KL-penalty* variant exists.)

Strengths: simple to implement, first-order (works with Adam), robust, good sample efficiency for an on-policy method, reuses data over several epochs, a strong default baseline.
Weaknesses: still on-policy (less sample-efficient than off-policy SAC/TD3), sensitive to ε / GAE / batch hyperparameters, clipping is a heuristic without TRPO's monotonic-improvement guarantee.

11. Derive the DDPG/TD3 algorithms. What are their strengths and weaknesses? (we may ask about the pseudo-code).

DDPG (Deep Deterministic Policy Gradient) (Lillicrap et al., 2015) is an off-policy actor-critic for **continuous** action spaces — essentially "DQN for continuous actions." It learns a deterministic policy $\mu_\theta(s)$ and a critic $Q_\varphi(s,a)$.

Derivation (deterministic policy gradient). For a deterministic policy the policy gradient is:

$$\nabla_\theta J = \mathbb{E}_{s\sim D}\Big[\nabla_a Q_\phi(s,a)\big|_{a=\mu(s)}\,\nabla_\theta \mu_\theta(s)\Big]$$

Reading it term by term.

- $\mu_\theta(s)$ — the **deterministic** actor: outputs one action, not a distribution.
- $\nabla_a Q_\varphi(s,a)|_{a=\mu(s)}$ — how the critic's value changes as you wiggle the action, evaluated at the actor's current output.
- $\nabla_\theta \mu_\theta(s)$ — how that action changes as you wiggle the actor's parameters.
- The two are chained (chain rule); $\mathbb{E}_{s\sim D}$ averages over replay states.

Intuition: "move the actor so it outputs actions the critic scores higher" — follow the critic's action-gradient uphill. This works **only** because actions are continuous and Q is differentiable in a (no discrete argmax).

i.e. move the actor to output actions that the critic rates highly (chain rule through Q). The critic is trained with the **DQN-style TD target** using target networks θ′, φ′:

$$y = r + \gamma Q_{\phi'}\big(s', \mu_{\theta'}(s')\big), \qquad L(\phi) = \mathbb{E}\big[(Q_\phi(s,a) - y)^2\big]$$

Reading it term by term.

- $\mu_{\theta'}(s')$ — the **target actor** picks the next action; primes denote slowly-updated target networks.
- $Q_{\varphi'}(s', \cdot)$ — the **target critic** evaluates it.

- $y = r + \gamma\, Q_{\varphi'}(s', \mu_{\theta'}(s'))$ — the bootstrap target. Note the discrete "max over actions" of DQN is replaced by "the action the actor outputs" — there is no max in a continuous action space.
- $L(\varphi) = \mathbb{E}[\,(Q_{\varphi}(s,a) - y)^2\,]$ — fit the critic to y by squared error.

Intuition: this is exactly DQN's regression, adapted to continuous control by letting the actor supply the next action in place of an argmax.

DDPG uses a **replay buffer** (off-policy), **target networks** with soft updates ($\theta' \leftarrow \tau\theta + (1-\tau)\theta'$), and **exploration noise** added to actions (originally Ornstein-Uhlenbeck, later Gaussian).

TD3 (Twin Delayed DDPG) (Fujimoto et al., 2018) fixes DDPG's chronic **overestimation bias** and brittleness with three tricks:

1. **Clipped double Q-learning:** two critics Q_{φ_1}, Q_{φ_2}; the target uses the **minimum**, $y = r + \gamma \min_i Q_{\varphi'_i}(s', \tilde{a}')$ — counters overestimation.
2. **Delayed policy updates:** update the actor (and targets) **less frequently** than the critic (e.g. every 2 critic steps) — lets the value estimate settle.
3. **Target policy smoothing:** add clipped noise to the target action $\tilde{a}' = \mu_{\theta'}(s') + \text{clip}(\varepsilon, -c, c)$ — regularizes Q, avoids exploiting sharp peaks.

Strengths: sample-efficient (off-policy + replay), handle continuous control well, TD3 is much more stable and higher-performing than DDPG. **Weaknesses:** DDPG is notoriously **unstable and hyperparameter-sensitive**; deterministic policies explore poorly (need injected noise); TD3 is more complex; both can still be brittle compared to SAC, which adds entropy for robustness.

12. Describe the replay buffer technique? Where is it used?

A **replay buffer** is a (usually fixed-size, FIFO) memory storing past transitions (s, a, r, s′, done). During learning, minibatches are **sampled (uniformly or by priority)** from the buffer instead of using only the most recent transition.

Why it helps:

- **Breaks temporal correlations** — consecutive transitions are highly correlated; random sampling produces more i.i.d.-like minibatches, stabilizing SGD.
- **Data efficiency / reuse** — each expensive environment interaction is used in many gradient updates.
- **Smooths the data distribution** over many past behaviors, reducing oscillation/forgetting.

Requirement: it stores data from old (behavior) policies, so it is only valid for **off-policy** algorithms. **Used in:** DQN and all its variants, DDPG, TD3, SAC. **Prioritized Experience Replay** samples transitions with probability proportional to their TD error (importance-

corrected) to focus on the most informative/surprising transitions. **Hindsight Experience Replay (HER)** relabels goals to learn from failed episodes (sparse-reward, goal-conditioned tasks).

13. What is double Q-learning? Why and when is it used?

Problem. Standard Q-learning's target uses $\max_{a'} Q(s',a')$. Because Q is noisy, the *max of noisy estimates* is systematically **biased upward** (maximization bias / overestimation): we use the same values both to **select** and to **evaluate** the best next action.

Double Q-learning (van Hasselt, 2010) decouples selection from evaluation using two estimators Q_A, Q_B. One picks the action, the other evaluates it:

$$a^* = \arg\max_{a'} Q_A(s', a'); \qquad \text{target} = r + \gamma\, Q_B(s', a^*)$$

Reading it term by term.

- Q_A, Q_B — two **independently** learned value estimators.
- a = *arg max*$_{a'}$ $Q_A(s',a')$ *— use* Q_A *to select** the best next action.
- target = r + γ Q_B(s', a) *— use the* other *estimator* Q_B *to evaluate** that chosen action.

Intuition: the max's upward bias arises from using the *same* noisy values both to pick and to judge the action. Selecting with Q_A but evaluating with Q_B makes the two noises **independent**, so they largely cancel instead of reinforcing.

(and symmetrically, randomly updating one or the other). Because the noise in selection and evaluation is independent, the upward bias largely cancels.

Double DQN is the deep version: reuse DQN's existing networks — the **online** net selects the argmax action, the **target** net evaluates it:

$$y = r + \gamma\, Q\big(s',\ \arg\max_{a'} Q(s', a'; \theta);\ \theta^-\big)$$

Reading it term by term.

- arg $\max_{a'}$ Q(s',a'; θ) — the **online** network θ **selects** the next action.
- Q(s', · ; θ^-) — the **target** network θ^- **evaluates** that selected action.
- y = r + γ × (that evaluation) — the corrected bootstrap target.

Intuition: the deep version of double Q-learning, reusing DQN's two existing networks — online selects, target evaluates — so it cancels most of the overestimation at essentially **zero extra cost**.

When/why used: whenever Q-learning-style **max-bootstrapping** with function approximation causes overestimation that degrades or destabilizes learning — i.e. essentially all value-based deep RL. It improves stability and final performance at negligible cost and is a component of Rainbow. The same idea (taking a **min** of two critics) appears in TD3 and SAC.

14. What are methods of ensuring exploration in policy algorithms?

- **Stochastic policies:** sampling actions from $\pi_\theta(\cdot|s)$ inherently explores; key in REINFORCE/A3C/PPO.
- **Entropy regularization / bonus:** add $\beta \cdot H(\pi(\cdot|s))$ to the objective to keep the policy from collapsing to determinism prematurely (A3C, PPO, SAC — where entropy is built into the objective itself, *maximum-entropy RL*).
- **Action / parameter noise:** add Gaussian or Ornstein-Uhlenbeck noise to actions (DDPG/TD3), or perturb network parameters (**NoisyNets**, parameter- space noise) for state-dependent, temporally-consistent exploration.
- **ε-greedy / Boltzmann (softmax)** action selection in value-based methods.
- **Optimism / intrinsic motivation:** count-based bonuses, curiosity (prediction-error rewards, ICM/RND), information gain — drive the agent toward novel states (good for sparse rewards).
- **Posterior sampling / Thompson-style:** bootstrapped DQN, randomized value functions.
- **Goal-based exploration:** HER, Go-Explore, diversity objectives.

15. Derive the SAC algorithms. What are their strengths and weaknesses? (we may ask about the pseudo-code).

SAC (Soft Actor-Critic) (Haarnoja et al., 2018) is an **off-policy maximum-entropy** actor-critic for continuous control. It augments the reward with the policy entropy, so the agent maximizes reward **and** acts as randomly as possible:

$$J(\pi) = \sum_t \mathbb{E}\big[r(s_t, a_t) + \alpha\, H(\pi(\cdot \mid s_t))\big]$$

Reading it term by term.

- $r(s_t,a_t)$ — the usual reward.
- $H(\pi(\cdot|s_t))$ — the **entropy** (randomness) of the policy at s_t.
- α — the **temperature**, weighting entropy against reward.
- $\Sigma_t\ \mathbb{E}[\,\cdot\,]$ — the summed expected value over the trajectory.

Intuition: maximize reward **and** stay as random as possible — the agent earns a bonus for keeping its options open, which drives exploration and robustness. As α → 0 this collapses back to ordinary RL.

α is the **temperature** trading off reward vs. entropy.

Soft value functions. The soft Q and V satisfy modified Bellman equations:

$$Q(s,a) = r + \gamma\, \mathbb{E}_{s'}\big[V(s')\big], \qquad V(s) = \mathbb{E}_{a\sim\pi}\big[Q(s,a) - \alpha \log \pi(a \mid s)\big]$$

Reading it term by term.

- *Left:* soft Q has the ordinary shape — immediate reward plus discounted next-state value.
- *Right:* the soft V averages Q over actions but **subtracts** α log π(a|s).
- −α log π(a|s) — the per-action entropy contribution; averaged over a ∼ π it equals +α H(π).

Intuition: the value of a state now includes a **bonus for acting randomly**. Folding the entropy term into the Bellman backup yields "soft" value functions whose greedy policy is a **Boltzmann distribution** over Q-values rather than a hard argmax.

Components / derivation.

- **Critics:** two Q-networks (clipped double-Q, as in TD3) trained on the soft TD target $y = r + \gamma(\min_i Q_{\varphi'_i}(s',\tilde{a}') - \alpha \log \pi(\tilde{a}'|s'))$, with $\tilde{a}' \sim \pi$.
- **Actor:** improve π by minimizing KL to the exponentiated soft-Q, equivalently maximize $\mathbb{E}_{a\sim\pi}[\min_i Q(s,a) - \alpha \log \pi(a|s)]$. The **reparameterization trick** $a = f_\theta(\varepsilon; s)$ (e.g. tanh-squashed Gaussian) gives a low-variance pathwise gradient.
- **Automatic temperature tuning:** α is adjusted to hold the policy entropy near a target $\bar{H}$ (a constrained-optimization dual).

Strengths: sample-efficient (off-policy + replay), very **stable and robust** to hyperparameters, strong exploration from the entropy term, state-of-the-art on continuous control, principled stochastic policy. **Weaknesses:** more moving parts (two critics, target nets, temperature); originally for continuous actions (discrete variants exist); entropy weighting and tanh-squashing add complexity; maximum-entropy objective can slightly bias the optimal policy if α is set wrong.

16. Describe briefly the maximum entropy RL formalism (compare with the "standard" approach).

Standard RL maximizes expected return only: $J(\pi) = \mathbb{E}[\Sigma\, \gamma^t\, r_t]$. The optimal policy is (in general) **deterministic**.

Maximum-entropy RL adds an entropy term so the agent prefers high-reward *and* high-randomness behavior:

$$J(\pi) = \sum_t \mathbb{E}_{(s_t,a_t)\sim\pi}\left[r(s_t,a_t) + \alpha\, H(\pi(\cdot \mid s_t))\right]$$

Reading it term by term.

- Same as the SAC objective, but with the expectation written explicitly over (s_t,a_t) generated by following π.
- r — reward; $\alpha\, H(\pi(\cdot|s_t))$ — entropy bonus with temperature α.

Intuition: the general **maximum-entropy RL** objective. Whereas standard RL (just $\Sigma\, \mathbb{E}[r]$) has a *deterministic* optimum, adding α H makes the optimum **stochastic** — a softmax over soft Q-values, $\pi^*(a|s) \propto \exp(Q_{soft}/\alpha)$. α dials from "purely greedy" ($\alpha \to 0$) toward "purely random" ($\alpha \to \infty$).

with $H(\pi(\cdot \mid s)) = -\mathbb{E}_{a\sim\pi}[\log \pi(a \mid s)]$ and temperature α (as $\alpha \to 0$ it recovers standard RL). The optimal policy is **stochastic** — a Boltzmann distribution over soft Q-values: $\pi^*(a \mid s) \propto \exp(Q_{soft}(s,a)/\alpha)$.

Why it's useful.

- **Better exploration** — the policy keeps trying multiple good actions instead of collapsing early.
- **Robustness** — captures multiple modes / near-optimal solutions; more resilient to model and environment perturbations.
- **Improved stability and transfer**, and a smoother optimization landscape.
- Connects RL to **probabilistic inference** ("control as inference"): the value backup uses a *soft* max (log-sum-exp) instead of a hard max.

This formalism underlies **Soft Q-Learning** and **SAC**.

✷ Exploration-exploitation

1. Multi-armed bandits model.

A **multi-armed bandit (MAB)** is the simplest exploration-exploitation problem — a one-state MDP. There are K **arms** (actions). Pulling arm a yields a stochastic reward drawn from an unknown fixed distribution with mean μ_a. At each round t = 1…T the agent picks an arm A_t and observes reward $X_{A_t,t}$.

The goal is to **maximize cumulative reward** over T rounds, i.e. learn which arm is best ($\mu^* = \max_a \mu_a$) while not wasting too many pulls on suboptimal arms. The central tension: **exploit** the arm that currently looks best vs. **explore** less-tried arms that might be better. Because there is a single state and no transitions, bandits isolate the exploration problem from the credit-assignment problem of full RL. Variants: stochastic, adversarial, contextual (state-dependent), Bayesian.

2. Regret and regret decomposition.

Regret measures the loss from not always playing the optimal arm. The (expected) cumulative regret after T rounds is:

$$R_T = T\mu^* - \mathbb{E}\Big[\sum_{t=1}^{T} \mu_{A_t}\Big] = T\mu^* - \mathbb{E}\Big[\sum_{t} X_t\Big]$$

Reading it term by term.

- $\mu^* = \max_a \mu_a$ — *the mean reward of the best** arm.
- $T\mu^*$ — what an oracle earns by always pulling that best arm for T rounds.
- $\mathbb{E}[\Sigma \mu_{A_t}]$ — expected reward of the arms the algorithm *actually* chose (A_t is the arm picked at round t).
- Their difference is R_T, the **regret**.
- The second form uses X_t, the observed reward, which equals μ_{A_t} in expectation.

Intuition: regret is the **shortfall versus an oracle** that always plays the best arm. Minimizing regret is the same as maximizing reward, just measured relative to the best achievable.

Minimizing regret is equivalent to maximizing reward, but regret is the natural *relative* performance measure (vs. an oracle that always plays the best arm).

Regret decomposition. Let $\Delta_a = \mu^* - \mu_a$ be the **suboptimality gap** of arm a, and $N_a(T)$ the number of times arm a is pulled. Then:

$$R_T = \sum_a \Delta_a \, \mathbb{E}\big[N_a(T)\big]$$

Reading it term by term.

- $\Delta_a = \mu - \mu_a$ — *the gap*: how much worse arm a is than the best.
- $N_a(T)$ — how many times arm a was pulled in T rounds; $\mathbb{E}[N_a(T)]$ is the expected count.
- The regret is Σ_a (gap) × (expected pulls).

Intuition: each pull of a suboptimal arm costs exactly its gap Δ_a, so total regret is those costs summed. To keep regret small, pull large-gap arms rarely — which is why every regret proof reduces to bounding $\mathbb{E}[N_a(T)]$ for suboptimal arms.

This is fundamental: regret is the sum over arms of (gap) × (expected pulls). It shows that to keep regret small an algorithm must pull each suboptimal arm only **O(log T)** times — and pull arms with larger gaps even less. All regret analysis reduces to bounding $\mathbb{E}[N_a(T)]$ for suboptimal arms.

3. Formulate asymptotic and worst case lower bounds on regret.

Asymptotic (instance-dependent) lower bound — Lai & Robbins (1985). For any "consistent" algorithm, on a fixed bandit instance,

$$\liminf_{T\to\infty} \frac{R_T}{\log T} \geq \sum_{a:\,\Delta_a>0} \frac{\Delta_a}{\mathrm{KL}(p_a \,\|\, p^*)}$$

Reading it term by term.

- $R_T/\log T$ — regret measured in units of log T; liminf is its long-run floor.
- $\Sigma_{a:\,\Delta_a>0}$ — sum over the suboptimal arms.
- Δ_a — the gap (cost per pull).
- $\mathrm{KL}(p_a \,\|\, p)$ — *the KL divergence between arm a's reward distribution and the best arm's: how* statistically distinguishable* they are.
- Δ_a / KL — a costly arm (large Δ) that is hard to tell apart from the best (small KL) is the most expensive to learn about.

Intuition: no consistent algorithm can beat this **Ω(log T)** floor. You must sample a suboptimal arm enough times to become statistically sure it is worse, and the harder it is to distinguish, the more pulls that takes. UCB and Thompson sampling attain this bound.

where $KL(p_a \| p^*)$ is the Kullback–Leibler divergence between arm a's reward distribution and the optimal arm's. So **no algorithm can achieve regret of smaller order than log T** (an Ω(log T) lower bound) on a fixed instance; the constant is governed by how statistically distinguishable each suboptimal arm is from the best. UCB and Thompson sampling match this bound (asymptotically optimal).

Worst-case (minimax / problem-independent) lower bound. Taking the worst instance for a given horizon, no algorithm can guarantee better than

$$R_T = \Omega\left(\sqrt{KT}\right)$$

Reading it term by term.

- K — number of arms; T — horizon.
- Ω(√(KT)) — a lower bound that holds for the **worst-case** instance, no matter the gaps.

Intuition: the log-T bound assumes fixed, constant-sized gaps; but if the gaps are tiny (on the order of 1/√T) the arms are nearly indistinguishable, and even the best algorithm suffers √(KT) regret. This is the **minimax** regime; MOSS and minimax-tuned UCB meet the matching O(√(KT)) upper bound.

(for K arms over T rounds). Intuitively when gaps are tiny (~1/√T) the arms are hard to tell apart. Algorithms like **MOSS** and minimax-tuned UCB achieve the matching O(√(KT)) upper bound. So there is a gap between the *log T* instance-dependent regime and the *√(KT)* worst-case regime — both are tight.

4. Discuss optimality properties of the bandit algorithms.

Several notions of "optimal" coexist:

- **Asymptotic optimality:** the algorithm's regret matches the Lai–Robbins constant, $R_T/\log T \to \Sigma\, \Delta_a/KL(p_a \| p^*)$. UCB (KL-UCB) and Thompson sampling are asymptotically optimal.
- **Minimax optimality:** worst-case regret matches the Ω(√(KT)) lower bound up to constants (MOSS, minimax UCB).
- **Order optimality:** achieves O(log T) instance-dependent regret (the correct *order*) even if the constant is not exactly Lai–Robbins-optimal (vanilla UCB1).
- **Bayesian optimality:** for a prior over bandit instances, the **Gittins index** policy is exactly optimal for discounted infinite-horizon bandits (but computationally heavy and brittle to the discounting assumption).

- **PAC / best-arm identification:** "pure exploration" — find an ε-optimal arm with probability ≥ 1−δ using as few samples as possible (different objective from regret).

Good algorithms are simultaneously order-optimal *and* (near) asymptotically optimal. A practical theme: **optimism in the face of uncertainty** (UCB) and **probability matching** (Thompson sampling) both achieve these guarantees.

5. Define UCB and discuss its properties.

UCB (Upper Confidence Bound) implements *optimism in the face of uncertainty*: keep, for each arm, a high-probability **upper bound** on its mean and always play the arm with the largest upper bound. The bound = empirical mean + exploration bonus that shrinks as the arm is sampled more.

UCB1 (Auer et al., 2002): after each arm is tried once, at round t play

$$A_t = \arg\max_a \left[\hat{\mu}_a + \sqrt{\frac{2 \ln t}{N_a(t)}} \right]$$

Reading it term by term.

- $\hat{\mu}_a$ — the **empirical mean** reward of arm a so far (the *exploitation* term).
- $\sqrt{(2 \ln t / N_a(t))}$ — the **exploration bonus** / confidence width:
- ln t (numerator) grows slowly with time, so arms not tried for a while look tempting again;
- $N_a(t)$ (denominator) shrinks the bonus as arm a is sampled more (we grow confident in it).
- $\arg\max_a$ — play the arm with the highest **upper bound** (mean + bonus).

Intuition: "**optimism in the face of uncertainty**" — treat each arm as good as its data plausibly allow. The bonus (from Hoeffding's inequality) guarantees no arm is starved forever, since its width keeps growing until it is re-sampled.

- $\hat{\mu}_a$ = empirical mean (exploitation term).
- The bonus $\sqrt{(2 \ln t / N_a)}$ grows with t (revisit arms we are unsure about) and shrinks with N_a (confidence in well-sampled arms). It comes from Hoeffding's concentration inequality.

Properties.

- **Logarithmic regret:** $R_T \leq O(\Sigma_{a:\Delta_a>0} (\ln T)/\Delta_a)$, so it is order-optimal; **KL-UCB** attains the exact Lai–Robbins constant (asymptotically optimal).
- **Deterministic** given the data, no tuning of an exploration schedule, no prior needed.
- Automatically balances exploration/exploitation; never permanently abandons an arm (every arm's bonus eventually grows).

- Extends to structured/contextual settings (**LinUCB**) and to RL (UCB-style bonuses for exploration). Slightly conservative; performs a bit worse empirically than Thompson sampling.

6. Define Bayesian multi-armed bandits. Characterize Bernoulli with Beta prior bandits.

Bayesian bandits put a **prior** over each arm's unknown reward parameter and update it to a **posterior** as rewards arrive; decisions use the full posterior (not just a point estimate). This naturally quantifies uncertainty.

Bernoulli–Beta (conjugate) bandit. Each arm gives reward 1 (success) with unknown probability $\theta_a \in [0,1]$, 0 otherwise. Put a **Beta(α_a, β_a)** prior on θ_a (Beta(1,1) = uniform is a common start). Because the Beta is conjugate to the Bernoulli likelihood, after observing outcomes the posterior is again Beta — updates are trivial counting:

$$\text{success: } (\alpha, \beta) \leftarrow (\alpha + 1,\ \beta) \qquad \text{failure: } (\alpha, \beta) \leftarrow (\alpha,\ \beta + 1)$$

Reading it term by term.

- (α, β) — the parameters of the **Beta posterior** on an arm's success probability θ.
- On a **success** (reward 1): increment α.
- On a **failure** (reward 0): increment β.

Intuition: because the Beta prior is **conjugate** to the Bernoulli likelihood, updating the belief is just **counting** — α−1 successes and β−1 failures seen so far. The posterior mean α/(α+β) tracks the empirical success rate, and the distribution **narrows** as pulls accumulate. This posterior is exactly what Thompson sampling draws its samples from.

So $\alpha_a - 1$ counts successes and $\beta_a - 1$ counts failures; the posterior mean is $\alpha_a/(\alpha_a+\beta_a)$ and its variance shrinks as the arm is pulled more (the distribution concentrates). This posterior is exactly what **Thompson sampling** samples from. Conjugacy makes the Bayesian update exact and O(1).

7. Discuss Thompson sampling.

Thompson sampling (TS) (Thompson, 1933) is a Bayesian, randomized strategy implementing **probability matching**: play each arm with the probability that it is the best, given current beliefs.

Algorithm (per round):

1. For each arm a, **sample** a parameter $\tilde{\theta}_a$ from its current posterior (e.g. Beta(α_a, β_a) for Bernoulli).

2. **Play** the arm with the highest sampled value, $A_t = \text{argmax}_a \tilde{\theta}_a$.
3. Observe the reward and **update** that arm's posterior.

The randomness of sampling provides exploration automatically: arms with uncertain (wide) posteriors occasionally sample high and get explored; as an arm is pulled its posterior narrows and stops being over-selected.

Properties.

- **Asymptotically optimal:** matches the Lai–Robbins log-T bound (proved by Agrawal & Goyal, Kaufmann et al.).
- **Strong empirical performance** — usually beats UCB in practice.
- Simple, naturally handles **contextual** and structured bandits, and extends to RL (**posterior sampling for RL / PSRL**).
- **Caveats:** needs a posterior (prior modeling + sampling), which can be expensive for complex models; performance can degrade if the prior/likelihood is badly misspecified.

⬢ Model-based methods

1. What is model-based RL, compare with model-free. What are its strengths and weaknesses?

Model-based RL learns (or is given) a model of the environment — the dynamics $\hat{P}(s'|s,a)$ and reward $\hat{R}(s,a)$ — and uses it to **plan**: simulate future trajectories to choose actions or to generate synthetic training data. **Model-free RL** skips the model and learns values/policies straight from real experience.

Strengths of model-based:

- **Sample efficiency** — a learned model produces unlimited simulated experience, so far fewer real interactions are needed (crucial when real data is expensive, e.g. robotics).
- **Planning / look-ahead** — can reason about the future (MCTS, MPC), adapt to new goals/reward functions without re-learning, and transfer.
- **Reusability** of the model across tasks.

Weaknesses:

- **Model bias / compounding errors** — small one-step errors accumulate over a rollout; planning can exploit model inaccuracies, giving poor real-world performance.
- **Harder to learn** accurate models of complex, high-dimensional, stochastic environments.
- **Computational cost** of planning at decision time.
- Often **lower asymptotic performance** than model-free when data is plentiful.

Hybrids (Dyna, MBPO, MBVE, world models) combine both: use the model for efficiency while grounding in real data for asymptotic performance.

2. Describe the Dyna-algorithm (DynaQ).

Dyna (Sutton) integrates **learning, planning, and acting** in one loop by using real experience for *both* (a) direct RL updates and (b) learning a model, then using the model to generate simulated experience for *extra* updates.

Dyna-Q loop, each real step:

1. **Act:** in state S choose A (ε-greedy on Q), observe R, S′.
2. **Direct RL (learn):** Q-learning update on the real transition: $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_a Q(S',a) - Q(S,A)]$.
3. **Model learning:** store Model(S,A) ← (R, S′) (deterministic model = a table of observed transitions).

4. **Planning:** repeat n times — sample a previously seen (S,A), retrieve (R,S′) from the model, and apply the **same Q-learning update** on this simulated transition.

Each real step thus drives n+1 value updates, dramatically speeding convergence. **Dyna-Q+** adds an exploration bonus κ√(time since (S,A) last tried) to the simulated reward, encouraging revisiting stale state-actions — useful in **changing** environments. Dyna shows the value of planning with a learned model even when the model is simple/tabular.

3. What is the back-propagation-through time algorithm? Why is it not used?

If we have a **differentiable** model of the dynamics, we can in principle plan by unrolling it for H steps, summing rewards, and **backpropagating the return through the entire chain of model and policy** to get analytic gradients ∂(Σr)/∂θ — like training an RNN with BPTT.

Why it is generally not used (in pure form):

- **Exploding / vanishing gradients** — the long product of Jacobians over the horizon causes gradients to blow up or vanish, exactly the RNN problem; long horizons are numerically unstable.
- **Chaotic / ill-conditioned dynamics** — many physical systems amplify tiny perturbations, so the gradient landscape is extremely sharp and noisy; tiny state changes give huge gradient swings.
- **Model errors compound** and the gradients flow through those errors, amplifying model bias.
- **Stochasticity** — environments and policies are stochastic; differentiating through sampled trajectories needs reparameterization and still gives high-variance gradients.
- **Local optima / non-convexity** of the unrolled objective.

So instead people use **shooting / sampling-based planning** (random shooting, CEM, MPPI), **MCTS**, or value-gradient methods that bootstrap (e.g. **SVG**) and only backprop through short horizons, mixing in learned value functions to avoid long unrolls.

4. Derive the LQR and iLQR algorithms. What are their strengths and weaknesses?

LQR (Linear-Quadratic Regulator) is the classic optimal-control solution when the dynamics are **linear** and the cost is **quadratic**:

$$x_{t+1} = Ax_t + Bu_t, \qquad \text{cost} = \sum_t \left(x_t^\top Q\, x_t + u_t^\top R\, u_t\right)$$

Reading it term by term.

- x_t — the **state** vector; u_t — the **control** (action) vector.
- $x_{t+1} = A\,x_t + B\,u_t$ — **linear** dynamics: A says how the state drifts on its own, B how the control moves it.
- $x_t^\top Q\,x_t$ — a quadratic penalty for being away from the origin ($Q \geq 0$ weights which state components matter).
- $u_t^\top R\,u_t$ — a quadratic penalty on control **effort** ($R > 0$).

Intuition: "steer the state to zero while spending as little control as possible," with everything measured by squared (quadratic) costs. This linear-dynamics + quadratic-cost structure is exactly what makes the optimal controller solvable in closed form.

(x = state, u = control/action, $Q \geq 0$, $R > 0$). **Derivation by dynamic programming (backward Riccati recursion):** the cost-to-go is quadratic, $V_t(x) = x^T P_t x$. Starting from the terminal $P_T = Q$ and going backward, substitute the dynamics into the Bellman equation and minimize over u_t. The optimum is a **linear feedback law** $u_t = -K_t x_t$ with

$$K_t = (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A$$

Reading it term by term.

- P_{t+1} — the **cost-to-go** matrix one step later (the value function is $V_{t+1}(x) = x^\top P_{t+1} x$).
- $B^\top P_{t+1} A$ — how a control applied now couples, through the dynamics, to future cost.
- $(R + B^\top P_{t+1} B)^{-1}$ — invert the **effective** control cost: immediate effort R plus the downstream cost the control creates.
- K_t — the **feedback gain**; the optimal control is $u_t = -K_t x_t$.

Intuition: the gain weighs spending control now (R) against the future cost a control incurs (via B and P_{t+1}). At runtime you just multiply the current state by $-K_t$ to get the best action.

$$P_t = Q + A^\top P_{t+1} A - A^\top P_{t+1} B\,(R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A$$

Reading it term by term.

- Computes P_t (cost-to-go now) from P_{t+1} (cost-to-go next), running **backward** from the terminal condition $P_T = Q$.
- Q — the immediate state cost.
- $A^\top P_{t+1} A$ — the future cost *if no control were applied*.

- $-A^\top P_{t+1}B\,(R+B^\top P_{t+1}B)^{-1}B^\top P_{t+1}A$ — the cost **reduction** the optimal control achieves (note it embeds the same factor that defines K_t).

Intuition: this is the **backward Riccati recursion** — the dynamic-programming backup specialized to linear-quadratic problems. "Value now = immediate cost + best achievable future cost." Iterating it from the horizon back to t = 0 produces every P_t, and hence every gain K_t.

iLQR (iterative LQR) extends LQR to **nonlinear** dynamics and general costs by iterating: (1) roll out the current control sequence; (2) **linearize** the dynamics (Taylor, Jacobians A_t,B_t) and take a **quadratic approximation** of the cost around the trajectory; (3) solve the resulting LQR for a control *update* (a backward pass giving feedback gains, then a forward pass with line search); (4) repeat to convergence. It is a Gauss-Newton / shooting method; **DDP** is the second-order variant that also uses dynamics Hessians.

Strengths: very **sample/computation-efficient**, exact for linear-quadratic systems, produces stabilizing feedback controllers, foundation of **MPC**, works great when an analytic or learned local model is available. **Weaknesses:** require a (locally) **known differentiable model**, only **local** optimality (iLQR linearizes around a trajectory), poor for highly nonlinear/contact-rich or high-dimensional discrete problems, can diverge without regularization/line search.

5. Compare open-loop vs closed-loop algorithms.

- **Open-loop:** plan a whole sequence of actions in advance and execute it **blindly**, without using new observations. Optimizes $a_0,a_1,\ldots,a_H$ once. Simple, but fragile — any disturbance, stochastic transition, or model error is never corrected, so error accumulates. Example: precomputed trajectory, naive trajectory optimization executed as-is.
- **Closed-loop:** the action depends on the **current observed state** via a policy/feedback law $a_t = \pi(s_t)$; the agent **re-decides using feedback** at each step, correcting for noise and model error. More robust and the right framing for stochastic environments. Examples: any RL policy, LQR feedback law, and **MPC** (Model Predictive Control), which plans open-loop over a horizon but **re-plans every step** using the latest state — giving closed-loop behavior from repeated open-loop optimization.

In **deterministic, perfectly modeled** problems open- and closed-loop optima coincide; under **stochasticity or model error**, closed-loop is strictly better because it can react. The practical lesson: even if you plan open-loop, **replan frequently** (MPC) to recover closed-loop robustness.

6. Describe the MCTS algorithms. What are their strengths and weaknesses? (we may ask about the pseudo-code).

Monte-Carlo Tree Search (MCTS) is a model-based planning algorithm that incrementally builds a search tree of states/actions, focusing computation on the most promising lines via sampled rollouts. Each iteration has **four phases:**

1. **Selection:** from the root, descend the tree by a **tree policy** (typically **UCT** = UCB applied to trees):

$$a = \arg\max_a \left[Q(s,a) + c\sqrt{\frac{\ln N(s)}{N(s,a)}} \right]$$

Reading it term by term.

- Q(s,a) — the current estimated value of action a at tree node s (**exploitation**).
- N(s) — number of visits to node s; N(s,a) — visits to the edge (s,a).
- √(ln N(s) / N(s,a)) — the UCB **exploration bonus**, larger for children that have been tried less often.
- c — a constant setting the exploration/exploitation balance.
- arg $\max_a$ — descend toward the child with the best optimistic score.

Intuition: this is **UCB1 applied inside the search tree** (“UCT”). At each node, follow the move that looks best after adding an uncertainty bonus for rarely-tried moves, so search effort concentrates on promising lines without permanently ignoring the rest.

balancing exploitation (Q) and exploration (the bonus). 2. **Expansion:** add a new child node for an untried action at the leaf. 3. **Simulation (rollout):** play out to a terminal/horizon using a default policy (random or learned) to get a return estimate. 4. **Backpropagation:** propagate the result back up, updating N and Q for all visited nodes.

After a budget of iterations, act greedily (most-visited / highest-value root action). In **AlphaGo Zero / AlphaZero**, rollouts are replaced *entirely* by a learned **value network** (the original **AlphaGo** still combined Monte-Carlo rollouts with the value net), and a learned **policy network** guides selection (PUCT), making MCTS far stronger.

Strengths: anytime (improves with more compute), needs only a generative model/simulator (no gradients), handles **large branching factors** by selective search, asymptotically optimal, naturally combines with learned value/policy nets, excellent for discrete deterministic games (Go, chess, shogi). **Weaknesses:** needs a **model/simulator**; expensive (many rollouts per decision); struggles with **large/continuous action spaces** and **long horizons** / sparse rewards; rollout/value quality bounds performance; less natural for stochastic high-dimensional control.

7. Describe the World models/Simple algorithms. What are their strengths and weaknesses?

World Models (Ha & Schmidhuber, 2018) learn a compact generative model of the environment and train the agent **inside its own "dream."** Three parts:

- **V (Vision):** a **VAE** compresses each high-dimensional observation (image) into a small latent vector z.
- **M (Memory):** an **MDN-RNN** predicts the next latent z′ (and reward) given z and action — a learned recurrent dynamics model with stochasticity.
- **C (Controller):** a tiny linear policy over (z, RNN hidden state), trained by a black-box optimizer (**CMA-ES**). Notably the agent can be trained *entirely in the latent dream* and transferred back.

SimPLe (Simulated Policy Learning) (Kaiser et al., 2019) applies the same idea to Atari: learn an **action-conditioned video-prediction** model of the game in pixel space, then train a policy (PPO) **inside the learned simulator**, periodically collecting a little real data to refine the model. It reaches good scores with ~100k real environment interactions (the "Atari 100k" benchmark; ≈400k frames at frame-skip 4) — **far more sample-efficient** than model-free DQN.

Strengths: strong **sample efficiency**; learning in latent/imagined space is cheap and parallelizable; compresses high-dim observations; enables planning and transfer.
Weaknesses: performance is **bounded by model fidelity**; models accumulate error and can be **exploited** by the policy ("hallucinated" reward); training the generative model is hard and unstable for complex visuals and long horizons; pixel-space prediction is expensive.
(Successors: **Dreamer** / Dreamer-v2/v3 plan in latent space with actor-critic.)

8. Describe the I2A algorithm.

I2A (Imagination-Augmented Agents) (Weber et al., 2017) combines model-based "imagination" with model-free RL while being **robust to model errors**. Instead of trusting a learned model to plan, I2A **learns to interpret** imagined rollouts as extra context for a policy.

Pipeline:

- An **environment model** (learned, action-conditioned) is rolled out from the current state for several steps under a **rollout policy**, producing **imagined trajectories** of predicted states and rewards.
- A **rollout encoder** (an RNN) processes each imagined trajectory **backward** into a summary vector — crucially this lets the agent *learn how much to trust* the model and extract useful information even from imperfect predictions.

- Multiple imagined rollouts (one per action, say) are aggregated and **concatenated with a model-free path** (features from the real observation).
- A final policy/value head outputs the action.

Key idea / strength: because the imagination is just *additional input* to a model-free agent (not a hard planner), I2A **degrades gracefully** when the model is imperfect — unlike pure planning, which an inaccurate model can wreck. It improved data efficiency and performance on Sokoban and other domains. **Weakness:** complex architecture, expensive imagination, still needs a decent model.

9. Describe the MBMF algorithm.

MBMF (Model-Based + Model-Free) (Nagabandi et al., 2018) is a hybrid that uses a learned model for **fast, sample-efficient bootstrapping** and then hands off to model-free RL for **high asymptotic performance**.

Two stages:

1. **Model-based control.** Learn a **neural-network dynamics model** $\hat{s}_{t+1} = f_\theta(s_t, a_t)$ from data (trained to predict the state *change*). Use it with **MPC + random shooting** (or CEM): at each step sample many candidate action sequences, simulate them through the model, pick the first action of the best sequence, execute, re-plan. This learns competent behavior with **very few real samples** but plateaus below expert level (MPC + model error caps performance).
2. **Model-free fine-tuning. Distill** the MPC controller into a neural policy via supervised imitation (DAgger-style), then **initialize a model-free RL algorithm** (e.g. TRPO) with that policy and continue training on real environment interaction to reach high asymptotic performance.

Strengths: combines model-based **sample efficiency** with model-free **asymptotic performance**, avoiding the weaknesses of each alone. **Weaknesses:** multi-stage pipeline; the MPC stage is computationally heavy at run time; model quality still limits the warm-start.

10. Describe the MBVE algorithm.

MBVE / MVE (Model-Based Value Expansion) (Feinberg et al., 2018) improves a model-free critic's **TD targets** using short model rollouts, without committing to long, error-prone planning.

Idea: instead of a 1-step TD target $r + \gamma V(s')$, **unroll the learned model H steps** and bootstrap the value only at the end:

$$V^{\text{targ}}(s) = \sum_{k=0}^{H-1} \gamma^k \hat{r}_k + \gamma^H V(\hat{s}_H)$$

Reading it term by term.

- $\hat{r}_k$, $\hat{s}_k$ — rewards and states **imagined** by rolling the *learned model* forward from s.
- $\Sigma_{k=0}^{H-1} \gamma^k \hat{r}_k$ — the discounted model-predicted rewards over H steps.
- $\gamma^H V(\hat{s}_H)$ — bootstrap on the learned value, but only at the *end* of the H-step rollout.

Intuition: the same shape as an n-step return, except the "real" rewards are replaced by **model predictions** for H steps — the horizon over which the model is still trustworthy — after which we lean on the learned value. Trusting the model only briefly yields lower-variance, more accurate targets while **capping model bias to the horizon H**.

where $\hat{r}_k$, $\hat{s}_k$ come from the model. By trusting the model only for a **short horizon H** (where it is accurate) and using the learned value beyond that, MBVE gives **lower-variance, more accurate targets** for an off-policy actor-critic (e.g. DDPG), improving sample efficiency while limiting model-bias to the horizon H. **STEVE** later makes H adaptive by interpolating horizons weighted by their (ensemble-estimated) uncertainty.

Strengths: better targets → faster, more sample-efficient learning; bounded exposure to model error; easy to bolt onto existing model-free methods. **Weaknesses:** still sensitive to model accuracy near the horizon; fixed H is suboptimal (hence STEVE); extra compute for rollouts.

11. Describe the TreeQN algorithm/architecture.

TreeQN (Farquhar et al., 2018) is a neural architecture that **embeds tree-structured planning into the Q-network itself** — model-based look-ahead made fully differentiable and trained end-to-end with model-free RL.

Instead of computing Q(s,a) with generic MLP layers, TreeQN builds a **recursive tree** in a learned **latent (abstract) state space**:

- **Encoder:** map observation → latent state z.
- **Transition model:** a learned function predicts the next latent state z_a for each action a (and a predicted reward).
- **Recursive expansion:** unroll this transition model to a fixed depth d, forming a planning tree of latent states.
- **Value backup:** apply a **differentiable tree backup** (a soft/max aggregation of predicted rewards plus leaf values) up the tree to produce Q(s,a).

The whole tree is one differentiable computation graph, trained with standard **DQN-style** losses — there is **no separate model-learning objective**; the latent model is shaped purely to make Q accurate. **ATreeC** is the actor-critic variant. **Strength:** combines planning's inductive bias with model-free training and generalization; better than plain DQN on box-pushing/Atari with the same data. **Weakness:** fixed shallow depth; the latent model is not grounded so it may not be a faithful environment model; tree width grows with action space.

12. Describe the UPN algorithm/architecture.

UPN (Universal Planning Networks) (Srinivas et al., 2018) learns a **latent space and dynamics model in which gradient-based planning works**, trained end-to-end via **imitation** so that the planner reproduces expert behavior.

Mechanism:

- Observations are encoded into a latent space; a learned **latent forward dynamics model** predicts how latent states evolve under actions.
- An **inner planner** performs **gradient descent through this differentiable model** to optimize an action sequence that drives the latent state toward a **goal latent** (goal-conditioned planning) — this is a differentiable trajectory optimizer unrolled inside the network.
- The **outer loop** trains the encoder + dynamics so that the *planned* actions match expert demonstrations (imitation loss backpropagated **through the planning process** — "learning to plan").

Because the representation is shaped to make planning succeed, the learned latent space is a useful, **goal-distance-like metric**, and the planner **generalizes** to new goals and even transfers (the learned representation/reward can be reused for RL on new tasks, longer horizons, different morphologies). **Strength:** end-to-end learned planner with strong generalization/transfer. **Weakness:** requires expert demonstrations; differentiable planning is expensive and can be unstable.

13. Describe selected three RL benchmarks.

- **Arcade Learning Environment (ALE) / Atari 2600.** ~57 classic video games from raw pixels with a shared discrete action interface. The standard testbed for **discrete-action, high-dimensional perception** RL (DQN, Rainbow, IMPALA). Tests generality across diverse games, sparse rewards, and long horizons; reported as human-normalized scores.
- **MuJoCo / OpenAI Gym continuous-control** (HalfCheetah, Hopper, Walker2d, Ant, Humanoid). Physics-simulated locomotion tasks with **continuous state and action** spaces and dense rewards. The standard benchmark for policy-gradient / actor-critic continuous

control (TRPO, PPO, DDPG, TD3, SAC). (DeepMind **Control Suite** is a similar standardized set.)

- **StarCraft II (SC2LE / PySC2)** and **DeepMind Lab / DMLab-30.** Large-scale, **partially observed**, long-horizon, multi-agent / 3D first-person navigation environments testing memory, hierarchy, and exploration at scale (AlphaStar, IMPALA).

(Other notable benchmarks: **Procgen** for generalization, **MetaWorld** for multi-task/meta-RL, **MineRL** for sample-efficient learning from demonstrations, **bsuite** for diagnostic behaviors.)

❖ Advanced topics in RL

1. Describe Reinforcement Learning for Improving Agent Design.

This line of work (Ha, 2019, “Reinforcement Learning for Improving Agent Design”) argues the agent’s **body (morphology) is part of the policy** and should be **learned jointly** with the controller. Rather than fixing the robot’s geometry and only learning how to control it, the method makes the **design parameters** (limb lengths, sizes, etc.) **learnable** alongside the policy and optimizes both to maximize reward.

Key idea: embed the morphology parameters into the same end-to-end optimization (policy gradients / black-box optimization), so the agent co-adapts its physical form and its behavior. The result is that a **better-suited body makes the control problem easier** — co-designed agents outperform agents with a fixed hand-designed body, often discovering non-obvious morphologies (e.g. modified leg proportions) that are easier to control and achieve higher return. It demonstrates that **embodiment matters** and that design and control should not be separated.

2. Discuss open-endedness on the example of POET.

Open-endedness is the property of a process that **keeps generating novel, increasingly complex, and interesting challenges and solutions indefinitely** — like natural evolution — rather than converging to a single optimum. The goal is endless innovation, not a fixed objective.

POET (Paired Open-Ended Trailblazer) (Wang et al., 2019) is an algorithm that **co-evolves environments and the agents that solve them**. It maintains a growing population of **(environment, agent)** pairs and repeatedly:

1. **Generates new environments** by mutating existing ones (e.g. 2D bipedal-walker obstacle courses).
2. **Optimizes** each paired agent on its environment (via ES).
3. **Transfers** agents between environments — a policy trained elsewhere may solve a new environment better than its native agent (**stepping stones**).
4. **Filters** new environments to keep only those that are neither too easy nor too hard (a “minimal criterion” / novelty + learnability test).

This curriculum **emerges automatically**: environments get harder as agents improve, and transfer lets solutions to hard problems be reached via a chain of intermediate problems that **direct optimization could never solve**. POET illustrates how open-ended co-evolution can produce diverse, increasingly capable behaviors without a fixed target.

3. Discuss morphological approach of Learning to Control Self-Assembling Morphologies.

This work (Pathak et al., 2019, “Learning to Control Self-Assembling Morphologies”) studies agents made of **modular primitive limbs** that can physically **link up** to form larger composite bodies, controlled by a **modular, shared neural-network policy**.

Each limb is a simple module with its own controller; modules communicate via **messages passed along the graph** of how they are connected (a graph neural network / dynamic message-passing policy). Because the **same** policy is shared across all modules and communicates only locally, the collective can **self-assemble** into different morphologies and **generalize** to bodies of sizes and shapes never seen in training (**zero-shot generalization** to more limbs). The connectivity (the morphology) is itself dynamic.

Significance: it shows **modularity + local communication** as a powerful inductive bias for control — coordination and complex behavior emerge from simple, identical, communicating parts, yielding robustness and generalization across morphologies (related to swarm/ collective intelligence).

4. What are inductive biases, give definition and examples.

An **inductive bias** is the set of **assumptions** a learning algorithm uses to **generalize** from finite training data to unseen inputs — the prior preference for some solutions/hypotheses over others. Without inductive bias, generalization is impossible (no-free-lunch); the *right* bias for a problem dramatically improves sample efficiency and generalization, while a wrong one limits the attainable solutions.

Examples (architectural / relational, after Battaglia et al.):

- **Convolutional networks (CNNs): locality + translation invariance** — the assumption that useful image features are local and position-independent (weight sharing across space).
- **Recurrent networks / RNNs: temporal invariance / sequentiality** — sharing weights across time, assuming the same dynamics at each step.
- **Graph networks (GNNs): relational/permutation invariance** — entities and their pairwise relations matter; invariant to node ordering.
- **Attention/Transformers:** soft relational reasoning over sets.
- **Algorithmic biases:** regularization (preference for simplicity/Occam), weight decay, the discount factor and Bellman structure in RL, action repeats, symmetry constraints, and the choice of state/feature representation.

In RL, planning architectures (TreeQN, I2A, UPN) inject the **bias that the world has predictable dynamics worth modeling and searching**.

5. Relational inductive bias on the example of Relational Deep Reinforcement Learning.

A **relational inductive bias** assumes that what matters are **entities and the relations between them**, and that reasoning should be **invariant to the entities' order/identity**. It biases a model toward computing pairwise (or higher-order) interactions rather than treating the input as a flat vector.

Relational Deep Reinforcement Learning (Zambaldi et al., 2018) injects this bias via **self-attention** over entities derived from the scene:

- A CNN produces a feature map; each spatial cell is treated as an **entity**.
- A **multi-head self-attention** module (a relational block, like a Transformer) computes interactions between all pairs of entities, iteratively refining their representations based on relations.
- These relation-aware features feed the policy/value heads.

Because attention is **permutation-invariant** and explicitly models pairwise relations, the agent learns **relational reasoning** that **generalizes** better: on the **Box-World** and **StarCraft II mini-games** it solved tasks requiring multi-step relational planning and generalized to configurations (more objects/longer plans) beyond training. It shows that adding a relational module improves **sample efficiency, performance, and out-of-distribution generalization**, and the learned attention is interpretable (it attends to task-relevant related objects).

6. Discuss the topic of causality and adversarial examples (generally and in RL).

Causality vs. correlation. Standard supervised/RL learning exploits **statistical correlations** in the training distribution. But correlation is not causation: a model may latch onto **spurious features** that happen to correlate with the target/reward but are not its cause. Such models fail under **distribution shift / intervention**, because the spurious correlation breaks. **Causal** models capture the data-generating mechanism and stay valid under interventions (Pearl's do-calculus).

Adversarial examples are inputs perturbed by tiny, often imperceptible amounts that cause a model to make confident mistakes. They reveal that models rely on **non-robust, spurious features** rather than the true causal/semantic ones.

In RL the problem is sharper because of the **closed loop**:

- **Spurious correlations / causal confusion** — an agent (especially in imitation learning) can key on features correlated with the demonstrator's action but not causal, and fail when deployed (see Q8).
- **Distribution shift** — the agent's own actions change the state distribution; a policy relying on correlations encounters states it never trained on.
- **Adversarial attacks on policies** — small perturbations to observations (pixels) can make a trained agent take catastrophic actions; an adversary in the environment or other agents can exploit this.

The remedy direction: learn **causal/robust representations**, use interventions (varying confounders), test under shift, and adversarial training.

7. Describe briefly the causal calculus.

Causal calculus (Judea Pearl's **do-calculus**) is a formal framework for reasoning about **interventions** and answering causal queries from a combination of data and a **causal graph** (a directed acyclic graph encoding which variables cause which).

Core distinction: **seeing vs. doing.**

- $P(Y \mid X = x)$ — *observation* (conditioning): how Y is distributed among units where X happened to equal x.
- $P(Y \mid \mathbf{do}(X = x))$ — *intervention*: how Y would be distributed if we **forced** X to x, severing X's own causes.

These differ when there is **confounding**. Do-calculus provides **three inference rules** for manipulating expressions with do(·), which let one decide whether a causal (interventional) quantity is **identifiable** from observational data given the graph, and if so, **express it** in terms of observed distributions (e.g. via the **back-door** and **front-door** adjustment criteria — controlling for the right variables to block confounding paths). The hierarchy of causation (the "**ladder**"): (1) association (seeing), (2) intervention (doing), (3) counterfactuals (imagining "what if I had acted differently"). This matters for RL because policy evaluation/off-policy learning are fundamentally interventional ("what happens if I *do* this action?").

8. Casual confusion on the example of Causal Confusion in Imitation Learning

Causal Confusion in Imitation Learning (de Haan, Jayaraman, Levine, 2019) identifies a counterintuitive failure: in imitation/behavioral cloning, **access to more information can make the policy worse**, because the learner **confuses correlation with causation** about which features cause the expert's actions.

Classic example: a driving agent that sees a **dashboard brake indicator** which lights up *because* the expert brakes. Behavioral cloning finds that the indicator is highly **predictive** of braking and uses it as a shortcut — but it is a **consequence**, not a cause, of the action. At test time the policy waits for the indicator that only its own (correct) action would trigger, so it fails to brake (distributional-shift collapse). More inputs → more spurious correlates → worse.

Their solution: treat the problem causally. (1) Learn a policy over a **disentangled representation** with a learned **causal graph / mask** selecting which features influence the action; (2) **resolve** the correct causal model by **targeted intervention** — either querying the expert or executing in the environment and using the resulting reward/feedback to **disambiguate** which graph yields good control. A small amount of interaction/queries suffices to pick the causal model that ignores the spurious feature. The lesson: imitation must distinguish **causes** of expert actions from mere correlates.

9. Discuss End to End Learning for Self-Driving Cars approach (note the problems with imitation learning).

NVIDIA's **End-to-End Learning for Self-Driving Cars** (Bojarski et al., 2016) trained a **single CNN** to map **raw front-camera pixels directly to steering commands**, with **no hand-engineered** lane detection, path planning, or control modules. The network learned the whole driving pipeline from ~72 hours of human driving data and could follow lanes and roads.

It is essentially **behavioral cloning (imitation learning)**, and the paper / follow-ups highlight its characteristic **problems**:

- **Distribution shift / compounding error (covariate shift).** The policy is trained only on states the *expert* visited. Small errors take the car to **off-distribution** states (drifting toward the lane edge) it never saw in training; errors compound and the car can leave the road. (The classic **DAgger** problem.)
- **No recovery data.** Experts rarely make mistakes, so the data lacks examples of recovering from bad situations. NVIDIA's fix: **augment** with images from **left/right shifted cameras** labeled with corrective steering, teaching recovery — an explicit patch for covariate shift.
- **No reasoning about long-term consequences / reward** — pure imitation copies actions without optimizing an objective, so it can't outperform or generalize beyond the demonstrator, and can inherit demonstrator biases.
- **Causal confusion** (as in Q8) — may latch onto spurious correlates.

The takeaway: end-to-end imitation is powerful and simple but brittle under distribution shift; mitigations include data augmentation, DAgger (interactive expert), and mixing in RL.

10. Discuss random exploration on the example of Learning to Fly by Crashing.

Learning to Fly by Crashing (Gandhi, Pinto, Gupta, 2017) is a striking example of learning a control/perception policy from **large-scale random (self-supervised) exploration**, including **deliberately crashing** a drone to collect **negative** examples.

Method: fly a cheap drone around indoor environments executing **random trajectories** until it **collides**, ~11,500 crashes over many hours. Each trajectory is automatically **self-labeled**: image frames far from the collision are **"safe/go straight"** (positive), frames just before impact are **"near-collision/turn"** (negative). A **CNN** is trained as a binary classifier on these auto-collected labels. At deployment the network, run on left/right/center image crops, decides whether to **move forward or turn**, enabling reliable indoor navigation and obstacle avoidance.

Significance: demonstrates that **failure (crashing) is cheap, informative data**, and that **random exploration + self-supervision** can replace expensive human labels or hand-engineered perception. No simulator or human demonstrations needed — the agent collects its own large, diverse, real-world dataset including the crucial **negative** (collision) examples that supervised datasets usually lack. It exemplifies learning from one's own mistakes at scale.

11. Discuss the domain randomisation technique on the example of OpenAi Rubik's cube project.

Domain randomization (DR) trains a policy in **simulation** while **randomizing** many aspects of the simulator (physics parameters, visual appearance, dynamics) so that the **real world looks like just another random variation**. This bridges the **sim-to-real gap** without real-world training.

OpenAI's Rubik's Cube project (Solving Rubik's Cube with a Robot Hand, 2019) used a **Shadow Dextrous Hand** to manipulate and solve a Rubik's cube, with the control policy (PPO + LSTM) and vision trained **entirely in simulation**, then transferred zero-shot to the real robot. They randomized geometry, friction, masses, gravity, actuator gains, observation noise, visuals (lighting, textures), etc.

Their key innovation was **Automatic Domain Randomization (ADR):** instead of fixing randomization ranges by hand, ADR **automatically widens** the distribution of environments as the policy gets better — an **emergent curriculum** that keeps the task at the right difficulty and produces an ever-more-robust policy. The **LSTM-based policy** showed **emergent meta-learning / adaptation at test time**, coping with novel perturbations never explicitly trained (a rubber glove, tied fingers, pushing the cube).

Strengths: no real-world training data needed, safe, robust transfer. **Weaknesses:** huge compute, needs a reasonable simulator, randomizing too much can make learning hard or yield over-conservative policies.

12. Discuss general/universal random functions and the hindsight replay buffer technique.

Universal Value Function Approximators (UVFA) (Schaul et al., 2015) generalize value functions to be **goal-conditioned**: instead of V(s) or Q(s,a), learn **V(s, g)** / **Q(s, a, g)** — value as a function of both the state and a **goal** g. A single network thus represents a whole **family of value functions/policies**, one per goal, and can **generalize across goals** (interpolating to unseen goals). This enables a single agent to pursue many goals and transfer between them.

Hindsight Experience Replay (HER) (Andrychowicz et al., 2017) solves the **sparse-reward, goal-conditioned** learning problem using a simple, powerful relabeling trick. In sparse tasks the agent almost never reaches the desired goal, so it gets no learning signal. HER's insight: **every trajectory succeeds at *some* goal — the state it actually reached.** So, for each stored episode, HER **relabels** transitions with **achieved goals** (e.g. the final state, or future states along the trajectory) in addition to the original goal, and recomputes the (now non-trivial) reward. These relabeled transitions go into the replay buffer.

The agent thus **learns from failures**: even when it misses the intended goal, it learns how to reach the goals it *did* hit, and this knowledge transfers (via the UVFA) to the real goals. HER + an off-policy algorithm (DQN/DDPG) makes previously-unsolvable sparse robotic manipulation tasks (push, slide, pick-and- place) learnable without reward shaping. It only works with **goal-conditioned, off-policy** learning.

◉ Distributional RL

1. Formulate the distributional RL operators.

Distributional RL (Bellemare, Dabney, Munos, 2017) models the **full distribution** of the return Z(s,a) — a random variable whose expectation is the usual Q(s,a) — instead of only its mean. This captures the **intrinsic randomness** of returns (multimodality, risk), giving a richer learning signal.

The **distributional Bellman equation** is, in distribution:

$$Z^{\pi}(s,a) \overset{D}{=} R(s,a) + \gamma\, Z^{\pi}(S',A'), \qquad S' \sim P,\ A' \sim \pi$$

Reading it term by term.

- $Z^{\pi}(s,a)$ — a **random variable**: the *whole distribution* of returns from (s,a), whose mean is the ordinary Q(s,a).
- $\overset{D}{=}$ — "equal **in distribution**" (the two sides share a distribution, not a single value).
- R(s,a) — the random immediate reward.
- $\gamma\, Z^{\pi}(S',A')$ — the next pair's return distribution, scaled by γ.
- $S' \sim P$, $A' \sim \pi$ — next state from the dynamics, next action from the policy.

Intuition: the ordinary Bellman equation lifted to **distributions**: the return-from-here distribution equals reward plus a discounted, shifted copy of the return-from-next distribution. Taking the expectation of both sides collapses it back to the familiar scalar Bellman equation for Q.

The corresponding **operators** act on distributions:

- **Distributional Bellman (evaluation) operator** $\mathcal{T}^{\pi}$: applies reward shift, discount **scaling** (multiply the random return by γ), and mixing over next-state/action distributions. It is a **γ-contraction in the maximal Wasserstein metric** (so repeated application converges to Z^{π}).
- **Distributional Bellman *optimality* operator** $\mathcal{T}$: same but with a greedy action $a' = \operatorname{argmax}_{a'} \mathbb{E}[Z(s',a')]$. It is **not** a contraction in Wasserstein in general and its dynamics are less well-behaved (the mean still converges, but the distribution can be unstable).

Three operations transform a return distribution under a backup: **shift** (by reward r), **scale/contract** (by γ toward 0), and **mix** (convex combination over transitions). Practical algorithms differ in **how they parametrize the distribution** (categorical, quantiles) and **which metric/projection** they use to fit the target distribution.

2. Describe C51.

C51 (Categorical DQN, Bellemare et al., 2017) is the first successful deep distributional RL algorithm. It represents Z(s,a) as a **categorical distribution** over a **fixed set of N = 51 "atoms"** (support points) $z_0 \ldots z_{N-1}$ evenly spaced on $[V_{min}, V_{max}]$:

$$Z(s,a) = \sum_i p_i(s,a)\,\delta_{z_i}, \qquad p_i = \text{softmax outputs}$$

Reading it term by term.

- z_i — a **fixed grid of atoms** (support points) evenly spaced across $[V_{min}, V_{max}]$.
- δ_{z_i} — a spike (Dirac mass) sitting at atom z_i.
- $p_i(s,a)$ — the probability put on atom z_i, produced by a **softmax** head (so the p_i are nonnegative and sum to 1).
- $Z(s,a) = \Sigma_i\, p_i\, \delta_{z_i}$ — the return distribution approximated as weighted spikes on the fixed atoms.

Intuition: model the return distribution as a **histogram on a fixed grid** — the network only outputs the bar heights p_i. Learning shuffles probability mass between atoms; because the atom *locations* are fixed, the Bellman target (whose atoms $r + \gamma z_i$ land between the grid points) must be **projected** back onto the grid — the ΦT step.

The network outputs the **probabilities $p_i(s,a)$** for each (action, atom). Learning:

1. Compute the **distributional Bellman target**: take greedy action $a^* = \text{argmax}_a\, \Sigma\, z_i p_i(s',a)$; form the target atoms $r + \gamma z_i$.
2. **Projection (ΦT):** the shifted/scaled atoms $r + \gamma z_i$ generally fall **between** the fixed support points, so C51 **projects** the target distribution back onto the fixed atoms by distributing each atom's probability to its two nearest neighbors (after clipping to $[V_{min}, V_{max}]$).
3. **Loss:** minimize the **cross-entropy (KL divergence)** between the projected target distribution and the predicted distribution.

C51 outperformed DQN substantially on Atari. **Limitations:** fixed bounded support $[V_{min}, V_{max}]$ must be chosen in advance; the projection step is needed because the support is fixed; number of atoms is a hyperparameter. These motivate the **quantile** approach (QR-DQN/IQN), which fix the *probabilities* and learn the *locations* instead.

3. Describe Quantile DRL.

QR-DQN (Quantile Regression DQN) (Dabney et al., 2018) flips C51's parametrization: instead of fixing the **locations** (atoms) and learning the **probabilities**, it **fixes the probabilities** (N uniform quantile fractions, each 1/N) and **learns the locations** — the **quantile values** $\theta_1(s,a)\dots\theta_N(s,a)$ that estimate the distribution's quantiles.

Advantages of this design: the support is **adaptive/unbounded** (no need to pick V_{min}/V_{max}) and **no projection step** is required, because the target atoms can take any value.

Learning uses **quantile regression**: to estimate the τ-quantile, minimize the **asymmetric (pinball) loss**, which penalizes over- and under-estimates asymmetrically. Combined with the Huber loss for smoothness, this gives the **quantile Huber loss**. The targets are the distributional Bellman backups $r + \gamma\theta_j(s',a^*)$. Theoretically, QR-DQN **minimizes the Wasserstein distance** to the target distribution (which C51's KL projection does not), matching the metric in which the distributional operator is a contraction.

QR-DQN matched or beat C51 with a cleaner formulation, and set up **IQN**.

4. Describe IQN.

IQN (Implicit Quantile Networks) (Dabney et al., 2018) generalizes QR-DQN by learning the **full quantile function** as an **implicit, continuous** function of the quantile fraction τ, instead of a fixed set of N quantiles.

Mechanism:

- Sample quantile fractions $\boldsymbol{\tau \sim U([0,1])}$. The network takes τ (via a cosine/embedding of τ combined with the state features) and outputs the corresponding quantile value $\mathbf{Z_\tau(s,a)}$ — a **reparameterized sample** of the return distribution.
- Train with the same **quantile (Huber) regression** loss as QR-DQN, but over **sampled** τ, τ' pairs (target uses fresh samples), so the network learns the entire quantile function rather than a fixed grid.
- At decision time, estimate $Q(s,a) = \mathbb{E}_\tau[Z_\tau(s,a)]$ by averaging over K sampled τ, then act greedily.

Advantages: arbitrary resolution (sample as many quantiles as you like at inference — a flexible computation/quality trade-off), strictly **more expressive** than QR-DQN, better Atari performance, and it enables **risk-sensitive policies** by reshaping the sampling distribution of τ (e.g. emphasize low quantiles for risk-averse behavior — a **distortion risk measure**). IQN was the strongest distributional component going into Rainbow-style agents.

5. What are the components of Rainbow. Describe them.

Rainbow (Hessel et al., 2018) combines **six** independent DQN improvements into one agent, showing they are **complementary** and together give state-of-the-art Atari performance. The components:

1. **Double Q-learning (Double DQN).** Decouples action selection (online net) from evaluation (target net) to remove the **overestimation bias** of the max operator.
2. **Prioritized Experience Replay.** Samples transitions with probability $\propto |\text{TD error}|^{\omega}$ (with importance-sampling correction), focusing learning on **surprising/informative** transitions; improves data efficiency.
3. **Dueling Network Architecture.** Splits the Q-network into a **state-value V(s)** stream and an **advantage A(s,a)** stream, recombined as Q = V + (A − mean A). Learns state values efficiently when actions don't all matter.
4. **Multi-step (n-step) returns.** Uses n-step bootstrap targets ($\Sigma\gamma^{k}r + \gamma^{n}\max Q$) for **faster reward propagation** and a better bias-variance trade-off.
5. **Distributional RL (C51).** Learns the **full return distribution** (51 atoms, cross-entropy loss) instead of just the mean — a richer signal.
6. **Noisy Nets.** Adds **learnable parametric noise** to network weights for **state-dependent, self-annealing exploration**, replacing ε-greedy.

Ablations showed **prioritized replay, multi-step, and distributional** were the most important, but all six contribute. Rainbow exemplifies that careful **integration** of orthogonal improvements beats any single one.